

Provelt

A Prover's Toolkit

Survey snapshot

Repository	https://github.com/VladimirReshetnikov/ProveIt
Pinned commit	56a402bf82a9aab119fcbe3e96b991d387b221c5
Survey date	24 August 2026
Primary toolchain	Lean 4 4.32.0, Mathlib v4.32.0
Companion toolchain	Rocq/Coq >= 9.2 (development history includes 9.0.1)
License	MIT-0

Lean workspace

One pinned root Lake workspace, dozens of named libraries, project-local workspaces for focused checking, and explicit opt-in heavy targets.

Rocq federation

One root `_CoqProject`, independent ports where useful, vendored or submodule dependencies, and per-project audit modules.

Certificates and audits

Kernel proofs, reflected decisions, generated sparse identities, executable checkers, source transcriptions, research ledgers, and carefully separated conditional endpoints.

Vladimir Reshetnikov

Expanded replacement for `ProveIt_Walkthrough.tex`.

The document is self-contained and all repository links are pinned to the surveyed commit.

Abstract

ProveIt is not a single formalization with a single proof boundary. It is a subject-organized collection of independent theorem developments, proof-assistant ports, executable decision procedures, finite certificates, paper audits, generated algebraic identities, and research corpora. The root Lean workspace gives the repository a common integration surface, while project-local Lake files, the root `_CoqProject`, vendored dependencies, and support scripts retain the autonomy required by large or technically unusual developments.

This walkthrough explains the repository as it exists at commit 56a402bf. It starts with the build and trust architecture, then surveys every current top-level mathematical subject. Special attention is given to the areas that have changed most since the previous walkthrough snapshot: the completed Fabius-function formalization; the primitive-circle and exponential-sum library; the progression-avoiding-permutation and Gowers–Szemerédi paper suite; the executable and certificate layers for quintic and sextic radical solvability; the rapidly growing Lazard quintic development; and the increasingly explicit separation between proved, conditional, statement-only, data-certified, and heuristic claims.

The intended reader is a maintainer or technically serious reviewer. The emphasis is therefore not merely on “what theorem is present,” but on where its statement lives, what imports make it reachable, how it is checked, which code executes during verification, what assumptions an audit reports, and which nearby files are evidence rather than proofs.

The previous walkthrough was pinned to commit 675d0a2ff0fc624fba199e006049016ba806f40a and dated 13 August 2026. The surveyed main was 685 commits ahead of that point. This edition is consequently a replacement survey rather than a changelog pasted onto the old text.

What this document has and has not checked

The repository survey was performed against the pinned Git tree, its root manifests, public facades, project-local READMEs, status ledgers, audit modules, and representative implementation files. The LaTeX document itself was compiled and visually inspected. This is not a claim that every expensive Lean target, every Rocq file, every native certificate, and every external support script was re-executed while preparing the walkthrough. Build commands and theorem-status descriptions report the repository’s checked interfaces and documented boundaries; they are not substituted for those checks.

Contents

Abstract	2
I. The repository as a system	9
1. Snapshot discipline and orientation	10
1.1. Why a pinned walkthrough is necessary	10
1.2. The five project species	10
1.3. Top-level map	11
1.4. Local directory conventions	11
1.5. Reporting vocabulary	12
2. Workspace topology and import reachability	13
2.1. Root Lean workspace	13
2.2. The root facade is broad, not complete	13
2.3. Lean library registry by domain	14
2.4. The Ramsey native certificate target	14
2.5. Rocq workspace	15
2.6. Vended and submodule dependencies	15
3. Trust, computation, and evidence	16
3.1. The trusted base is project-specific	16
3.2. Common computation modes	16
3.3. Source-transcription boundaries	17
3.4. Generated algebraic shards	17
3.5. Conditional results are first-class artifacts	17
II. Mathematical content by subject	19
4. Algebra	20
4.1. Overview	20
4.2. The explicit Jacobian counterexample	20
4.2.1. The three-variable witness	20
4.2.2. Symmetry and the collision orbit	21
4.2.3. Stabilization and degree reduction	21
4.2.4. Audit route	22
4.3. Polynomial formulas through degree four	22
4.3.1. Algebraic branch discipline	22
4.3.2. Exact Gaussian-rational front end	22
4.3.3. Exact rational enclosures	23
4.3.4. Executable rational root approximations	23
4.4. Three Abel–Ruffini routes	23
4.4.1. The explicit rational quintic	23
4.4.2. The Selmer family over $\mathbb{Q}$	23
4.4.3. The generic fixed-field construction	24
4.5. Deciding radical solvability of an integer quintic	24
4.5.1. Mathematical decomposition	24
4.5.2. Primitive recursion and machine semantics	24
4.5.3. The 302-term coefficient table	24
4.5.4. Independent Rocq routes	25

4.6.	Deciding radical solvability of an integer sextic	25
4.6.1.	Reducible branch	25
4.6.2.	Irreducible branch	25
4.6.3.	Rocq MuRec implementation	25
4.7.	The Lazard solvable-quintic program	25
4.7.1.	Why this is a separate project inside PolynomialFormulas	25
4.7.2.	Corrected formula and rejected raw contracts	26
4.7.3.	Proof-carrying solver interface	26
4.7.4.	Root-origin route	26
4.7.5.	Generated invariant and elimination certificates	27
4.8.	Recommended algebra reading routes	27
4.8.1.	For a theorem consumer	27
4.8.2.	For a certificate auditor	27
4.8.3.	Focused builds	28
5.	Analysis	29
5.1.	Overview and an important documentation warning	29
5.2.	Trigonometric identities	29
5.2.1.	Golden-ratio sine sum	29
5.2.2.	Quadratic arctangent identity	29
5.3.	Three integral powers force an integral exponent	30
5.4.	The tiny exponent tower	30
5.5.	The two-base Alaoglu–Erdős program	31
5.5.1.	Statement and headline status	31
5.5.2.	Natural-candidate reduction	31
5.5.3.	Exact finite exclusion through $x < 14$	31
5.5.4.	Four Exponentials reduction	32
5.5.5.	Radical and algebraic reformulations	32
5.5.6.	Counting, equidistribution, and density	33
5.5.7.	Higher differences and exponential-polynomial rigidity	33
5.5.8.	Valuations, tropical blocks, and finite-place relations	33
5.5.9.	Orbit, carry, Kummer, and interpolation branches	34
5.5.10.	The weight-two collapse frontier	34
5.6.	The Fabius function	35
5.6.1.	Public scope	35
5.6.2.	Construction and uniqueness	35
5.6.3.	Exact dyadic evaluator	35
5.6.4.	Complete paper formalizations	35
5.6.5.	Finite q -binomial identities and the global binary-reduction series	36
5.6.6.	Negative Laplace product and periodic correction	36
5.6.7.	Trust and source status	36
5.7.	Analysis reading order	37
6.	Combinatorics	38
6.1.	Overview	38
6.2.	Derivative expansions and OEIS A290268	38
6.3.	A shared framework for power towers	39
6.3.1.	A000081: the syntax baseline	39
6.3.2.	A002845: towers of twos	39
6.3.3.	A199812: towers of ordinals	40
6.3.4.	A198683: principal powers of i	40
6.4.	Radical expressions and OEIS A158415	40
6.4.1.	Exact lexical semantics	41
6.4.2.	The size-15 certificate	41
6.5.	Klarner’s polyomino growth constant	41
6.6.	Perfect squared squares	42
6.7.	Ramsey theory and progression-avoiding permutations	42
6.7.1.	Common formal vocabulary	42
6.7.2.	Paper catalogues and proof layers	42

6.7.3.	Exact finite counting through 20	43
6.8.	Combinatorics reading routes	43
7.	Computability	44
7.1.	Overview	44
7.2.	Turing degrees	44
7.2.1.	Lean core	44
7.2.2.	Rocq extensions and explicit hypotheses	45
7.2.3.	Deliberate nonclaims	45
7.3.	Combinatory logic: SK, SKI, and Iota	45
7.3.1.	The source calculus and bracket abstraction	45
7.3.2.	SK, SKI, and the noninjective reverse compiler	45
7.3.3.	Pure Iota and its runtime	46
7.3.4.	Embedding Iota back into lambda terms	46
7.3.5.	Partial recursion and Turing machines	46
7.3.6.	Audits	46
7.4.	Busy Beaver	46
7.4.1.	Score and time are different functions	47
7.4.2.	Finite classification targets	47
7.4.3.	Domination results	47
7.5.	Computability reading order	47
8.	Logic	48
8.1.	Overview	48
8.2.	First-order logic	48
8.3.	Modal logic and the Foundation port	49
8.3.1.	FoundationPort	49
8.3.2.	Modal semantic core	49
8.4.	Propositional and equational projects	49
8.4.1.	Shared natural deduction	49
8.4.2.	Sheffer stroke and single axioms	50
8.4.3.	Equational certificate checker	50
8.5.	Peano arithmetic list coding	50
8.5.1.	Parity and audit discipline	50
8.6.	Bounded consistency of PA	51
8.6.1.	Numeralwise scheme	51
8.6.2.	Uniform internal theorem	51
8.7.	Interpretability	51
8.7.1.	PA and hereditary finite set theory	51
8.7.2.	The ZFC-in-PA program	51
8.8.	Undecidability, finite bases, and model theory	52
8.9.	Logic reading order	52
9.	Number theory	53
9.1.	Overview	53
9.2.	Fermat's Last Theorem for exponent four	53
9.3.	The primitive circle problem and exponential sums	53
9.3.1.	Source and transcription boundary	53
9.3.2.	Proved analytic infrastructure	54
9.3.3.	Statement-only deep estimates	54
9.3.4.	Recommended reading route	54
9.4.	Exact integer sums	55
9.5.	A rational floor orbit	55
9.6.	A PA sentence for an RH criterion	55
9.7.	Number-theory reading order	55
10.	Set theory	56
10.1.	Overview	56
10.2.	A first-order ZF base	56

10.3. Closure axiomatization	56
10.4. Bounded consistency of ZFC	56
10.5. Set-theory reading order	57
11. Ancillary material, tools, and repository boundaries	58
11.1. Research articles and source corpora	58
11.2. Support generators	58
11.3. Rocq compatibility tools	58
11.4. Vendored libraries	59
11.5. Projects moved out of the repository	59
III. Building, auditing, and extending the repository	60
12. Reproducing the surveyed workspace	61
12.1. Checkout and pin verification	61
12.2. Mathlib cache and root build	61
12.3. Choosing the right Lean command	61
12.3.1. Default integration build	62
12.3.2. Named library or facade	62
12.3.3. Direct module elaboration	62
12.3.4. Temporary import probe	62
12.4. Reading Lake's build plan	62
12.5. Rocq aggregate build	63
12.6. External tools are optional until a target says otherwise	63
13. Auditing theorem status and trust	64
13.1. Start from an endpoint theorem	64
13.2. Lean assumption audits	64
13.3. Rocq assumption audits	65
13.4. Audit a generated certificate in layers	65
13.5. Audit native decision separately	65
13.6. Audit paper catalogues	66
13.7. Audit conditional endpoints	66
14. Debugging large formal developments	67
14.1. Classify the failure before editing	67
14.2. Minimize the import surface	67
14.3. Coercion and normalization failures	67
14.4. Resource controls are part of the module contract	68
14.5. Generated-data drift	68
14.6. Native and linker failures	68
14.7. False statements and failed interfaces	68
15. Adding or expanding a project	69
15.1. Choose the canonical subject path	69
15.2. Define one public facade	69
15.3. Write a status ledger before the project becomes large	69
15.4. Design certificate interfaces for failure	70
15.5. Add audits as build products	70
15.6. Keep documentation reachability-tested	70
16. Current formalization frontiers	71
16.1. Completed headline developments	71
16.2. Precisely isolated open barriers	72
16.3. Architectural strengths	72
16.4. Architectural pressure points	72

IV. Reference	74
A. Command crib sheet	75
A.1. Root and broad subject builds	75
A.2. Logic and number-theory targets	75
A.3. Audits and searches	75
B. Project and facade index	77
C. Trust-boundary checklist	78
D. What changed since the previous walkthrough	79
E. Glossary of repository terms	80
F. Closing perspective	81

List of Tables

1.1.	Current top-level mathematical organization.	11
1.2.	Status words used throughout this walkthrough.	12
2.1.	Root Lean libraries.	14
3.1.	Computation and certificate modes in the repository.	16
4.1.	Algebra project status.	20
5.1.	Analysis project status.	29
16.1.	Major current barriers and their formalized reductions.	72
B.1.	Canonical project locations and first files to read.	77

Part I.

The repository as a system

1. Snapshot discipline and orientation

1.1. Why a pinned walkthrough is necessary

The repository moves too quickly for a prose guide that links only to main. A statement may remain true while its facade moves, a target may remain declared while becoming unreachable from a default build, a reflected certificate may acquire a native implementation, or a conditional endpoint may be strengthened without changing the surrounding topic name. Every file link in this document therefore points to commit 56a402bf82a9aab119fcbe3e96b991d387b221c5. The short hash in the footer is not decoration: it is the coordinate system for the claims on the page.

Four different notions of “present in the repository” must be kept separate:

1. A source file exists in the Git tree.
2. A facade imports the source file, so users of that facade can see it.
3. A Lake or Rocq target reaches the facade during a requested build.
4. The file exports a theorem with the advertised proof status, rather than merely a proposition-valued definition, a conditional theorem, a generated datum, or an exploratory result.

The old walkthrough often had enough context to infer these distinctions. The present repository is too large for inference to be safe, so this edition states them directly.

1.2. The five project species

Most directories fit one or more of the following species.

Theorem developments. Definitions and proofs culminate in named public theorems. Examples include the Jacobian counterexample, first-order completeness, the exact Klarner bound, and the Fabius-function formalization.

Paper formalizations and statement catalogues. A source paper is transcribed into definitions whose values are propositions. Completed formal proofs appear as separate theorems, conventionally with the suffix `_holds`. A catalogue may be complete even when its proof layer is not. The Gowers–Szemerédi tree makes this distinction especially visible.

Certificate developments. A mathematical argument reduces a large finite claim to data checked by a proved verifier, a reflected Boolean, sparse polynomial normalization, or a native decision procedure. The logical specification, the implementation used for execution, and the trust boundary of the reduction are separate artifacts.

Research and source corpora. OCR-corrected papers, scripts, Wolfram notebooks, generated tables, agent reports, and feasibility studies preserve provenance and guide future work. They are not theorem declarations merely because they sit next to formal code.

Infrastructure and ports. Common syntax, semantic bridges, coding libraries, compatibility shims, and vendored dependencies support several mathematical projects. Their correctness surface is often more important than their headline visibility.

When encountering a dramatic claim, ask: *What is the smallest public theorem that says it, and what has to be imported or executed to obtain that theorem?* The answer usually identifies both the real proof boundary and the right file to audit.

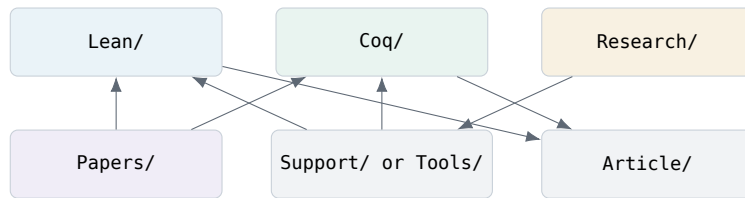
1.3. Top-level map

Table 1.1.: Current top-level mathematical organization.

Directory	Principal projects	Current role
Algebra	Jacobian conjecture; polynomial formulas	Explicit counterexamples, formulas through quartic, three Abel–Ruffini routes, exact polynomial solvers and approximators, recursive quintic/sextic radical decisions, and the Lazard quintic certificate program.
Analysis	Trigonometric identities; exponential identities; Fabius function	Small exact identities beside a large analytic/arithmetic formalization of the Fabius function, its dyadic evaluator, source papers, finite and infinite q-binomial identities, corrected asymptotics, and claim audits.
Combinatorics	Expression enumeration; derivative expansions; squared squares; polyominoes; Ramsey theory	OEIS semantics and certificates, a perfect squared-square witness, the exact 4.5235 Klarner bound, progression-avoiding permutation papers, finite counting certificates, and a growing formalization of Gowers’s proof of Szemerédi’s theorem.
Computability	Busy Beaver; combinatory logic; Turing degrees	Machine semantics and exact small-state values, weak-lambda universality through SK/SKI/Iota, and a broad set-degree development with an explicit Lean/Rocq coverage ledger.
Logic	Propositional, equational, first-order, modal, arithmetic, interpretability	Reusable calculi, completeness and compactness, correspondence theory, PA coding and undecidability, Presburger elimination, bounded consistency, and PA/HF or ZFC-in-PA interpretability projects.
NumberTheory	Diophantine equations; integer points; integer sums; rational enumeration; RH statement	FLT for exponent four, a large primitive-circle/exponential-sum library, exact summation and enumeration results, and an arithmetic sentence representing an RH criterion.
SetTheory	ZF; Closure axiomatization; bounded ZFC consistency	First-order set theory infrastructure, deductive and semantic equivalence of Closure with ZF, and numeral-wise bounded-complexity consistency for ZFC.
Tools	Rocq 9.2 compatibility	Small build-compatibility support; larger tools that grew up here have moved to their own repositories.
lib	Foundation, MathComp Abel, undecidability, synthetic computability, Coq-BB5	Third-party source only: Git submodules, pinned external trees, and a retained Busy Beaver certificate snapshot.
Oeis	Legacy A290268 location	A small compatibility/legacy tree; the integrated Lean library is under Combinatorics/DerivativeExpansions/A290268 .
Shenanigans	Redirect README	Deliberate kernel-loophole and paradox work moved to the separate Shenanigans repository .

1.4. Local directory conventions

A coherent project normally uses some subset of the following siblings:



These arrows are informational, not import edges. A paper transcription may determine a statement without being read by the kernel. A Python script may generate a sparse table that is later checked by a Lean theorem. An article may summarize a theorem without participating in its proof. The local README is expected to explain the intended direction of evidence.

Public Lean import surfaces conventionally live in a file matching the library name, such as `Analysis/FabiusFunction/Lean/FabiusFunction.lean`. Public Rocq developments often culminate in an `Audit.v`, a facade, or both. Generated files should be reachable from a checked aggregate, not merely present in a directory. Large research projects increasingly include a claim crosswalk or completion ledger so that “all statements are formalized” has a verifiable meaning.

1.5. Reporting vocabulary

Table 1.2.: Status words used throughout this walkthrough.

Label	What exists	What the label does not imply
proved	A theorem term checks in the stated assistant, modulo its printed assumptions and any declared computation boundary.	That another assistant has parity; that every stronger prose paraphrase follows; or that the theorem is in a default build.
conditional	A theorem proves the conclusion from an explicit hypothesis that remains undischarged.	That the hypothesis is known true, easy, or harmless. The ZFC-in-PA project records a previous false candidate hypothesis as a warning.
statement catalogue	A definition gives an exact proposition corresponding to a source claim.	That the proposition has a proof. Proposition-valued definitions add no axiom and prove nothing by themselves.
partial	A documented subset of the intended theorem family is proved, or one port has a stronger endpoint than another.	A vague percentage. The local ledger should name the residue.
generated	Data or source was produced by an external program.	That the generator is trusted. The important question is whether a proved checker validates the output.
Data-certified	A finite computation, interval partition, or external exact replay supports the claim.	Kernel proof unless a formal theorem consumes and validates the certificate.
Heuristic	Numerical or structural evidence guides the expected result.	A theorem, a certificate, or even a uniquely determined answer.

2. Workspace topology and import reachability

2.1. Root Lean workspace

The root workspace is pinned by `lean-toolchain` and `lake-manifest.json`. At the snapshot, both Lean and Mathlib are at 4.32.0. The package declaration in `lakefile.toml` has seven default targets:

```
ProveIt
PACoding
PACoding.Audit
PAUndecidable
PAUndecidable.Audit
BoundedPAConsistency
BoundedPAConsistency.Audit
```

This is an integration policy, not a ranking of mathematical importance. In particular, the broad `ProveIt` facade imports many subject facades but deliberately omits several expensive, specialized, conditional, or separately audited surfaces. Conversely, a library declaration can exist without appearing in `ProveIt.lean`.

The normal root bootstrap is:

```
lake exe cache get
lake build
```

The broad build is intentionally expensive. A maintainer should prefer a named library, a facade module, or a direct `lake env lean` invocation while editing one project.

2.2. The root facade is broad, not complete

`ProveIt.lean` imports the main public surfaces for algebra, analysis, combinatorics, computability, logic, number theory, and set theory. It includes the current Ramsey paper facades and `IntegerPoints`. It does not import every audit, every large certificate shard, every default target, or every conditional endpoint.

The practical consequences are:

- Success of `lake build ProveIt` establishes elaboration of the broad facade, not of every library declared in `lakefile.toml`.
- Success of plain `lake build` includes the seven default targets, which is a different set.
- A file that is not imported by a target can remain broken unnoticed by that target.
- A `lean_lib` with only a facade root may silently ignore an attempted dotted target that is not declared as a root or glob.

The ZFC-in-PA README documents that `lake build +ZFCinPA.Numerals0mega` exits successfully while building nothing, because the root `ZFCinPA` library exposes only its facade. The reliable one-module check is a direct elaborator invocation with explicit output paths, or adding the module to the facade. A successful `Lake` command is not evidence until the build plan reaches the intended module.

2.3. Lean library registry by domain

The root manifest declares the following integrated libraries. The leading + used in command examples requests a module/library as an explicit target even when it is not part of the default build.

Table 2.1.: Root Lean libraries.

Domain	Library names	Notes
Algebra	JacobianConjecture, PolynomialFormulas	The polynomial library contains both compact formula modules and a very large generated/certificate surface.
Analysis	TrigonometricIdentities, ExponentialIdentities, FabiusFunction	Fabius is now a completed large project, not a statement-only placeholder.
Expression enumeration	PowerTowers, A158415, A290268	The OEIS projects have different semantic domains and different finite-certificate boundaries.
Geometric combinatorics	SquaredSquare, KlarnerConstant	Squared-square certificates and a full geometric proof of the Klarner bound.
Ramsey/additive combinatorics	GowersSzemerédi, RamseyPaperCommon, RamseyCounterShim, RamseyPaperCertificates, DavisEntringerGrahamSimmons1977, Sharma2012, LeSaulnierVijay2010, LeSaulnierVijay2011	Includes a separately linked C implementation for the opt-in native finite-count target.
Computability	BusyBeaver, CombinatoryLogic, TuringDegrees	The broad Busy Beaver facade excludes large classifications.
First-order logic	FirstOrder, ArbitraryLanguageCompactness, QuantifierCommutation	Independent fixed-language completeness plus a Mathlib-backed arbitrary-language compactness target.
Propositional/equational logic	ShefferStroke, NaturalDeduction, FiniteMatrixNoncharacterizability, MonotonicityOfEntailment, PrincipleOfExplosion, EquationalLogic, BooleanAlgebra	Mostly small, focused, and independently buildable.
Arithmetic and interpretability	PAHF, NoFiniteModel, BoundedPAConsistency, PAListCoding, PAFiniteBasisReduction, Foundation, PAUndecidable, PresburgerArithmetic	The vendored Foundation tree is itself exposed as a Lean library.
Number theory	DiophantineEquations, IntegerSums, IntegerPoints, RationalEnumeration, RiemannHypothesis	IntegerPoints is large and currently Lean-only.
Set theory and ZFC-in-PA	ZF, BoundedZFCConsistency, ZFCinPA, ClosureAxiomatization	The bounded ZFC theorem and the uniform ZFC-in-PA target have deliberately different statuses.

2.4. The Ramsey native certificate target

The Ramsey suite illustrates why library metadata belongs in a mathematical walkthrough. The logical specification `reflectedM` is a direct finite enumeration of permutations satisfying a proved midpoint-free predicate. An `implemented_by` attribute installs `fastReflectedM` for native execution. For inputs through 20, that function calls an extern C routine from `Combinatorics/Ramsey/Lean/RamseyPaperCommon/ThreeFreeCounter.c`; beyond 20 it falls back to an unsafe Lean implementation of the same prefix-extension recurrence.

The separate `RamseyPaperCertificates` target links the shared library built through `RamseyCounterShim`. Its theorem `reflectedM_values_through_twenty` is discharged by `native_decide`. This gives an extremely compact checked theorem surface, but it is not an ordinary kernel normalization of the direct enumeration. The native compiler/runtime, the `implemented_by` replacement, and the reviewed C counter are part of that theorem’s computation boundary. The mathematical bridge from `reflectedM` to the paper-local counting functions is proved separately.

Specification versus executable replacement

The direct definition of `reflectedM` is the logical specification used by proofs. The fast native implementation is not proved extensionally equal to that definition inside Lean. This is a deliberate performance boundary, and the walkthrough treats it like the repository’s other visible `native_decide` boundaries rather than describing it as pure kernel reduction.

2.5. Rocq workspace

The root `_CoqProject` is a federation of logical roots rather than a mirror of `lakefile.toml`. It registers the source list and `-Q/-R` mappings for the projects that currently have Rocq developments, including the large PolynomialFormulas tree, modal logic, arithmetic codings, and vendored Busy Beaver certificates.

The normal aggregate build is:

```
git submodule update --init \
  lib/Coq-Synthetic-Computability \
  lib/MathComp-Abel
opam install --yes --deps-only \
  ./lib/MathComp-Abel/coq-mathcomp-abel.opam
pwsh -NoProfile -File \
  Computability/TuringDegrees/Coq/BuildSyntheticComputability.ps1
rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq
```

Not every Lean project has a Rocq mapping. In particular, the current Fabius, IntegerPoints, Klarner, A290268, Ramsey, bounded-ZFC, and ZFC-in-PA developments are not root Rocq projects. Conversely, the Rocq modal port and some independent semantic implementations do not have a simple one-to-one Lean sibling. Parity is therefore a theorem-by-theorem property, never a directory-wide default.

2.6. Vendored and submodule dependencies

`.gitmodules` pins four submodules:

- `lib/Coq-Library-Undecidability-current`;
- `lib/FormalizedFormalLogic-Foundation`;
- `lib/MathComp-Abel`;
- `lib/Coq-Synthetic-Computability`.

The `lib/Coq-BB5` tree is a retained source snapshot rather than one of those four Git submodules. The older `lib/Coq-Library-Undecidability` copy coexists with the current submodule for compatibility/history. The rule for auditing is simple: identify the exact imported path and revision, not merely the upstream project name.

3. Trust, computation, and evidence

3.1. The trusted base is project-specific

Saying “Lean checked it” or “Coq checked it” is only the beginning of a trust account. A useful account names:

1. the theorem statement and its assumptions;
2. the kernel and standard logical axioms reported by audit commands;
3. any compiler/runtime mechanism used to construct the proof term;
4. any unproved implementation replacement or foreign function used by reflected computation;
5. any manually transcribed finite table or source statement on which the theorem is parameterized;
6. any conditional premise that remains in the theorem signature.

The repository is unusually good at exposing these distinctions. It uses audit files, `##print axioms`, `Print Assumptions`, `coqchk`, status tables, and explicit theorem names such as `..._of_successorImplications`. The main maintenance risk is not hidden admissions; it is that a reader sees a nearby proved theorem and silently upgrades a stronger conditional or statement-only claim.

3.2. Common computation modes

Table 3.1.: Computation and certificate modes in the repository.

Mode	Typical use	Trust and review consequences
Kernel reduction / tactics	Algebraic normalization, finite small decisions, rewriting, exact arithmetic	The tactic constructs a proof term checked by the kernel. Source transcription and standard axioms may still matter.
<code>decide</code>	Medium finite propositions with a decidable instance	Reduction remains in the kernel; resource use can be high, but no native compiler boundary is introduced.
<code>native_decide</code>	Large finite identities, sparse tables, reflected enumeration	Includes Lean’s native compiler/runtime. If execution reaches an <code>implemented_by</code> or external path, that replacement also enters the boundary.
<code>Rocq vm_compute / conversion</code>	Finite certificates and reflected Booleans	Uses Rocq’s VM/conversion machinery, with the resulting term checked under the documented kernel model. Audit with <code>Print Assumptions</code> and <code>coqchk</code> .
Proved checker plus generated data	Sparse polynomial tables, interval partitions, combinatorial certificates	The generator is reproducibility tooling. Soundness rests on the checker theorem and the exact data imported into the assistant.
External exact replay	Python integer/rational arithmetic or Wolfram exact arithmetic	Valuable independent evidence and regression checking; not a kernel theorem unless formally connected.
Statement catalogue	Paper claim represented as a Prop-valued definition	Zero proof content by itself. Look for a companion <code>_holds</code> theorem and ensure the facade imports it.

Mode	Typical use	Trust and review consequences
Conditional compiler/bridge	A high-level endpoint derived from a semantic or syntactic transfer premise	Honest formal progress, but the premise is the open problem. Audit the premise for plausibility and for known countermodels.

3.3. Source-transcription boundaries

Several projects formalize published mathematics from scanned or OCR-derived documents. The assistant kernel does not compare a local definition with the source PDF. The repository handles this in three ways:

- corrected source transcriptions are committed under `Papers/` or `Article/`;
- discrepancies, typographical corrections, and unavailable originals are documented in a README or claim crosswalk;
- theorem names and statement catalogues provide a reviewable source-to-formal map.

The IntegerPoints project is a representative example: many deep results are present as exact proposition-valued statements derived from corrected transcriptions, while a substantial subset of the analytic machinery has actual `_holds` proofs. The distinction is part of the formal status, not an editorial footnote.

3.4. Generated algebraic shards

The PolynomialFormulas project contains hundreds of generated modules for large symmetric-polynomial and Dummit/Lazard identities. The correct audit path is not to review every coefficient visually. It is to identify:

1. the abstract identity the shard is intended to establish;
2. the generated sparse representation and normalization semantics;
3. the theorem that connects normalized data to the abstract polynomial construction;
4. whether the final finite equality uses `kernel decide`, `native_decide`, or another boundary;
5. the generator and its regression tests, which matter for maintainability even when they are not trusted for soundness.

This pattern turns a large computer-algebra calculation into a small trusted checker plus inspectable data. It also creates a real engineering obligation: facade reachability, shard completeness, deterministic regeneration, and failure-localized module structure are part of the proof architecture.

3.5. Conditional results are first-class artifacts

The repository contains several valuable conditional endpoints:

- the remaining A198683 $n = 12$ decision depends on a partition witness and, after the proved near-one split, one narrow overflow “no miracles” condition;
- the Lean BB4 transition-normal-form reduction is sound, while the exhaustive root equality remains conditional;
- the Rocq uniform PA bounded-consistency endpoint is conditional even though every externally indexed instance is proved;
- the ZFC-in-PA uniform sentence is conditional on eight successor implications at arbitrary, including nonstandard, indices;

- the principal primitive-circle theorems remain conditional on deep exponential-sum estimates and RH hypotheses represented in the formal statements.

A conditional theorem can be the strongest precise description of an open frontier. It becomes misleading only when its hypothesis is omitted from prose or buried behind a facade name. This walkthrough repeats the hypothesis boundary whenever the conclusion is likely to be quoted independently.

Part II.

Mathematical content by subject

4. Algebra

4.1. Overview

The algebra tree has only two immediate children, but one of them has become the repository’s largest concentration of formal and generated code.

Table 4.1.: Algebra project status.

Project	Status	Principal endpoint
Algebra/JacobianConjecture	proved	An explicit characteristic-zero polynomial map in dimension three has nonzero constant formal Jacobian determinant and a checked collision; stabilization refutes the conjecture in every dimension at least three.
Low-degree formulas	proved	Field-generic Lean formulas through quartic, independent Rocq checks, exhaustive fixed-size root collections, and exact Gaussian-rational solver interfaces.
Abel–Ruffini	proved	Rootwise rational obstruction for $X^5 - 4X + 2$, padded all-degree obstruction, a Lean Selmer family over $\mathbb{Q}$, and a generic symmetric-function construction.
Integer quintic decision	proved	A primitive-recursive Boolean deciding all-root radical solvability, with verified factor search, Frobenius–Dummit resolvent, Chapman correction, and Turing-machine existence.
Integer sextic decision	proved	A recursive decision using factor search and separating degree-15/degree-10 resolvents; computable but not claimed primitive recursive.
Lazard solvable-quintic formula	partial	Large proof-carrying and root-origin infrastructure, corrected identities, counterexamples to false raw contracts, generated invariant certificates, and a documented restart/crosswalk; the automatic complete public formula endpoint remains active work.

4.2. The explicit Jacobian counterexample

4.2.1. The three-variable witness

The project formalizes the map announced by Levent Alpöge on 19 July 2026. Write

$$\begin{aligned}
 P &= (1 + xy)^3z + y^2(1 + xy)(4 + 3xy), \\
 Q &= y + 3x(1 + xy)^2z + 3xy^2(4 + 3xy), \\
 R &= 2x - 3x^2y - x^3z.
 \end{aligned}$$

For $F = (P, Q, R)$, both Lean and Rocq prove the formal polynomial identity

$$\det J_F = -2.$$

They also verify explicit collisions. A denominator-free one is

$$F(-1, 1, 5) = F(0, -2, -16) = (0, -2, 0),$$

and the project proves the full rational one-parameter family

$$F(t, -1/t, 5/t^2) = F(0, 2/t, -16/t^2) = (0, 2/t, 0) \quad (t \neq 0).$$

This is stronger than two unrelated evaluations. The formal statement packages the Keller condition and noninjectivity in one counterexample, then derives the absence of a polynomial inverse. Lean’s public theorem is field-generic in characteristic zero before specialization to $\mathbb{C}$. Rocq independently defines a small multivariate-polynomial syntax, formal differentiation, and determinant evaluation over Coquelin’s complex field.

4.2.2. Symmetry and the collision orbit

The formalization does not treat the family as a bag of points. It proves the discrete equivariance

$$F(-x, -y, z) = (P, -Q, -R)(x, y, z)$$

and a weighted torus action with weights $(-1, 1, 2)$:

$$F(x/s, sy, s^2z) = (s^2P, sQ, R/s), \quad s \neq 0.$$

The rational family is then derived from one integral collision by scaling and mirroring. This organization matters for maintenance: the family lemmas are rewrites through a structural action, not repeated calls to polynomial normalization. It also explains why the parameter involution is $t \mapsto -t$.

4.2.3. Stabilization and degree reduction

Lean proves that adjoining untouched coordinates preserves the constant determinant and the collision. The public stabilization theorem therefore refutes the conjecture in every dimension $n \geq 3$, not merely at the original witness dimension.

A separate stable-equivalence program lowers the maximum ordinary degree. A checked five-variable representative has degree profile $(6, 6, 4, 3, 4)$, determinant -2 , and an integral collision. Its construction factors through two unitriangular transformations around $F \times \text{id}^2$, so determinant and collision preservation are transparent. Exact research certificates continue through maximum degrees five and four to a degree-three representative in ten variables. Degree three is globally optimal for a nonaffine Keller counterexample. The Lean/Rocq public developments cover the original and degree-six representatives; the full stable frontier and verification scripts live under [Algebra/JacobianConjecture/Research](#).

Formal theorem versus exact research frontier

The dimension-three refutation, its stabilization, symmetry, torus action, and the degree-six stable representative are proof-assistant theorems. The additional degree-five, degree-four, and ten-variable cubic representatives are exact reproducible research artifacts unless and until their corresponding formal modules are imported by the public development. The local README identifies that boundary explicitly.

4.2.4. Audit route

A compact audit proceeds in this order:

1. read [Algebra/JacobianConjecture/README.md](#) for the formulas and scope;
2. inspect the Lean and Rocq Counterexample, Scaling, CollisionFamily, and SimplerCounterexample modules;
3. run each Audit module and coqchk;
4. run the Python research verifiers only when reviewing the stable-degree frontier rather than the kernel theorem.

4.3. Polynomial formulas through degree four

4.3.1. Algebraic branch discipline

The low-degree developments avoid pretending that an arbitrary characteristic-zero field comes with total canonical radical functions. A square root is represented by an element s and an equation $s^2 = D$; a cube root by $u^3 = z$; Cardano's compatible branch condition and a primitive cube root of unity are explicit hypotheses. Ferrari's formula similarly receives the resolvent data needed for the difference-of-squares factorization.

This design has two advantages. First, the core correctness claims are algebraic and field-generic. Second, branch choices cannot disappear into an opaque analytic convention. Once certified radical values are supplied, executable fixed-size collections are returned, and biconditional theorems characterize exactly when an input is one of the roots.

The sequence of results is conventional but unusually complete:

- linear uniqueness;
- both quadratic branches, their factorization, and exhaustiveness;
- the Tschirnhaus translation of a cubic, Cardano's depressed-cubic equations, compatible radical pairs, and the three-factor identity using a primitive cube root of unity;
- quartic normalization and depression, the cubic resolvent, Ferrari factorization, and four-entry exhaustiveness.

The Coq cubic project includes a concrete complex primitive cube root, so the exhaustive theorem is not a vacuous implication over a real-only carrier.

4.3.2. Exact Gaussian-rational front end

`GaussianPolynomialSolver.solve` specializes the low-degree machinery to coefficients in $\mathbb{Q}[i]$. It accepts five coefficients, permits zero leading coefficients, dispatches through quartic, cubic, quadratic, linear, and constant cases, and returns exact `RadicalExpression` trees. A nonzero constant has no roots; the identically zero polynomial has a distinct result constructor.

Two endpoints describe the output:

- `solve_factorization` proves the exact ordered product, retaining multiplicities;
- `eval_eq_zero_iff_contains` proves soundness and exhaustiveness of membership in the returned finite list.

This is a total noncomputable mathematical function. Classical choice selects exact complex radicals using proved existence theorems. It is not code-generated numerical software and does not make a kernel primitive out of radical extraction.

4.3.3. Exact rational enclosures

Every `ExplicitRadical` supports `boundingBox`. Given rational $\varepsilon > 0$, the function returns rational endpoints for a complex rectangle. The specification proves containment of the exact value, ordering of both coordinate intervals, and width and height at most ε . Solver bridge theorems transfer these guarantees to returned polynomial roots.

This interface is particularly useful for consumers that need finite, inspectable rational output without weakening the exact algebraic object. The enclosure is proved about the exact radical value; it is not a floating-point post-processing step.

4.3.4. Executable rational root approximations

A distinct API, `GaussianPolynomialApproximation.approximations`, is fully executable. It accepts a nonzero polynomial of actual degree at most four and a rational $\varepsilon > 0$, returning a vector of Gaussian-rational points whose length is exactly the actual degree. A nonzero constant returns the empty vector.

The implementation normalizes the polynomial, searches for exact monic separable factor layers, and enumerates rational contraction-disk certificates. All runtime tests are finite Booleans over naturals and Gaussian rationals. The analytic existence argument appears only in erased termination proofs for `Nat.find`; no selected complex root or real-number operation occurs in the returned data computation.

The correctness theorem provides a position-matched exact root multiset, bounds Manhattan distance by ε , and preserves multiplicity. It is a correctness-first exhaustive search, not an optimized numerical root finder. Fast seed certificates cover several common low-degree examples.

4.4. Three Abel–Ruffini routes

The project intentionally keeps three negative theorems separate because their coefficient fields and quantifier scopes differ.

4.4.1. The explicit rational quintic

Both Lean and Rocq use

$$X^5 - 4X + 2.$$

Lean connects Mathlib’s Galois-theoretic obstruction to a local syntax of rational constants, field operations, and chosen roots. Rocq uses MathComp Abel’s `algterm` semantics and proves that individual root representability is preserved under automorphisms. Irreducibility makes all roots conjugate, upgrading a whole-root-set obstruction to a rootwise theorem: no complex root has a radical expression over $\mathbb{Q}$.

Multiplication by X^{n-5} gives a degree- n polynomial for every $n \geq 5$ with obstructed quintic roots. This refutes a universal *complete* radical formula at every such degree. The adjective is essential because the padded polynomial also has the easy root zero.

4.4.2. The Selmer family over $\mathbb{Q}$

Lean proves a stronger degree-exact rational family using

$$X^n - X - 1.$$

For every $n > 4$, the development proves monicity, exact degree, exactly n complex roots, irreducibility, and a Galois-group surjection onto the full symmetric group. Nonsolvability of S_n then places every root outside the radical closure of $\mathbb{Q}$. The public theorem is the rational all-roots result `SelmerAbelRuffini.every_degree_gt_four_has_rational_polynomial_with_no_radical_root`.

4.4.3. The generic fixed-field construction

A second Lean-only construction takes independent variables $x_0, \dots, x_{n-1}$, the rational-function field L_n , and its symmetric fixed field $K_n = L_n^{S_n}$. The orbit polynomial of x_0 has the n variables as distinct roots, generates L_n , and has Galois group S_n . For $n \geq 5$, every root is nonsolvable by radicals over K_n .

This is structurally clean and independent of Selmer’s arithmetic theorem, but its coefficient field is not $\mathbb{Q}$. The repository exposes both results rather than conflating “generic all-roots” with “rational all-roots.”

4.5. Deciding radical solvability of an integer quintic

4.5.1. Mathematical decomposition

The decision problem asks whether every complex root of one integer-coefficient quintic is expressible by radicals. The checked procedure follows the Galois-theoretic dichotomy:

1. Integral monicization normalizes the input without changing the relevant splitting-field property.
2. A bounded search detects linear and quadratic factors. Reducibility leaves factors of degree at most four, whose radical solvability is already formalized.
3. In the irreducible branch, the scalar Frobenius–Dummit sextic detects whether the Galois group lies in the solvable Frobenius group F_{20} .
4. Chapman’s correction handles repeated roots of the specialized resolvent, closing a gap in the naive distinct-root criterion.
5. A bounded rational-root search decides whether the resolvent has the required rational root.

Every search bound and coefficient transformation is proved sufficient. The result is not merely “decidable by general algebra”: a concrete Boolean characteristic function is connected to the exact all-root radical predicate.

4.5.2. Primitive recursion and machine semantics

The quintic Boolean is proved primitive recursive. The public layer supplies:

- semantic correctness of the assembled Boolean;
- `ComputablePred` for the all-root radical predicate;
- existence of a `Part recToTM2` program satisfying the encoded execution relation.

The theorem does not claim that the resulting machine has been printed as a standalone efficient solver. It proves that the verified components compose to a primitive-recursive decision and to a program in the formal machine model.

4.5.3. The 302-term coefficient table

The seven elementary-symmetric coefficient polynomials of the scalar resolvent are committed as a 302-term sparse table. A proved normalization procedure checks those data against the six abstract scalar roots over every commutative ring, then connects the table to the symmetric-polynomial construction. This makes the final Boolean directly evaluable rather than merely existential.

Four largest finite reductions use `native_decide`; three use ordinary `decide`. The native compiler/runtime is therefore part of the former certificates’ boundary. The generator scripts and tests are regeneration aids; the checked sparse-normalization theorem is the semantic bridge.

4.5.4. Independent Rocq routes

Rocq contains two complementary implementations:

- a direct canonical coefficient Boolean combining bounded factor search and the computed scalar resolvent, with a reflection theorem to the exact MathComp-Abel all-root predicate;
- an earlier semantic implementation through an abstract splitting field and a natural-number-code reflector.

The direct route is closer to an executable coefficient algorithm. The semantic route is valuable as an independent specification-level cross-check. Their coexistence is intentional, not redundant generated translation.

4.6. Deciding radical solvability of an integer sextic

4.6.1. Reducible branch

For exact degree six, a bounded search for monic factors of degrees one through three decides reducibility. A nonlinear factorization leaves only factors of degree at most four; a linear factor delegates the remaining quintic to the verified quintic criterion.

4.6.2. Irreducible branch

Two block-system resolvents, of degrees 15 and 10, separate the solvable transitive subgroups from the nonsolvable cases. Their integer coefficients are computed as sparse symmetric polynomials, and bounded rational-root tests decide the two conditions.

The separating parameters are found by a proved terminating unbounded search. Consequently the assembled Boolean is Computable, not claimed Primrec. The public endpoints include semantic correctness, ComputablePred, and existence of a verified PartrecToTM2 program.

4.6.3. Rocq MuRec implementation

The Rocq files named SexticMuRec... go beyond a semantic existence proof. They compile signed arithmetic, monicization, bounded factor and root searches, the quintic branch, and compact pair/triple collision evaluators to the explicit `recalg` language of Coq-Library-Undecidability. Genuine unbounded minimization selects the separating parameters after a proof of existence.

Public graph theorems establish MuRec_computable for both a seven-coefficient zigzag encoding and a single natural code. Corresponding decidability theorems return explicit sums $\{P\} + \{\neg P\}$ for the exact all-root predicate. An older MathComp-Abel semantic sextic decision remains as an independent cross-check.

4.7. The Lazard solvable-quintic program

4.7.1. Why this is a separate project inside PolynomialFormulas

The checked decision “is this quintic solvable by radicals?” does not itself produce Lazard’s explicit radical expressions for all roots. Conversely, a proof-carrying Lazard formula can be sound without automatically constructing its invariant and branch certificate for every solvable input. The repository maintains both lines and makes their interface explicit.

The current Lazard tree is best understood as six interacting layers:

1. general Fourier and resolvent algebra;
2. invariant theory for the F_{20} action, including Molien data, Reynolds operators, symmetric rational functions, and finite-module arguments;

3. corrected concrete quintic invariants and a displayed Gröbner/leading-term descent route;
4. large exact root-invariant relations and determinant/critical-elimination certificates;
5. radical branch reconstruction, Fourier numerators, coherent projections, and root recovery;
6. Gaussian-rational dispatcher and completeness specifications.

The module count is not accidental bloat. Large coefficient identities are split so that failures localize, native reductions remain bounded, and generated shards can be regenerated independently.

4.7.2. Corrected formula and rejected raw contracts

The formalization incorporates corrections to Lazard’s printed formula, including two erroneous terms in P_{22} and an exceptional sign choice required when the first determination would make a denominator vanish. It also records stronger negative findings:

- the unrestricted modular version of one invariant-theory theorem fails in a regular C_6 action over $\mathbb{F}_3$;
- later displayed theorems are false under the paper’s literal definition;
- equation-only invariant contracts admit extraneous tuples and are therefore insufficient for formula soundness.

Rather than hiding those failures behind strengthened hypotheses, the Lean/Rocq developments include explicit counterexamples and build corrected interfaces. Sound root reconstruction now requires cyclic Fourier evidence or origin from an actual labelled root tuple.

4.7.3. Proof-carrying solver interface

`GaussianQuinticSolver.solve` accepts six Gaussian-rational coefficients. If the quintic coefficient vanishes, it delegates to the complete degree-at-most-four solver. For a genuine quintic it classically selects a Lazard package when one exists. That package contains invariant equations, radical choices, denominator conditions, and proofs of the four cyclic Fourier identities for the concrete components produced by the formula.

The inverse-Fourier theorem then proves all five returned values are roots, and the values are reified as exact radical-expression trees. Because the current certificate excludes singular solvable inputs such as X^5 , a missing Lazard package may fall back to a classically selected `CompleteRadicalSolution`. Distinct result constructors preserve which route was used.

The public soundness theorem states that every returned expression satisfies the input polynomial. Completeness of the genuine automatic Lazard branch and constructive generation of its certificate remain active work. A noncomputable classical completeness specification is mathematically useful but should not be advertised as an executable general quintic solver.

4.7.4. Root-origin route

The newer route starts from an ordered complete root tuple, derives the invariant and Fourier relations from those roots, proves the corrected displayed numerator identities, and reconstructs the tuple. This removes the logical weakness of arbitrary equation-only certificates: the data are tied to an actual splitting configuration.

A denominator-safe alternate reconstruction recovers all four nonzero Fourier-mode fifth powers through coherent projection branches and adjoins four fifth roots. It avoids pretending that a single quotient replacement handles every $E = 0$ case. The dedicated `Algebra/PolynomialFormulas/LazardPaperClaimCrosswalk.md` and `Algebra/PolynomialFormulas/LazardQuinticFormalization.tex` distinguish source-complete, locally checked, facade-reachable, and still-open obligations.

4.7.5. Generated invariant and elimination certificates

The most visible recent growth is in exact certificate modules. They cover, among other things:

- finite invariant bases and Molien-series coefficients;
- symmetric-module and Artin-successor constructions;
- displayed Gröbner bases and leading-term descent;
- square, product, cube, and fifth-power root-invariant relations;
- determinant matrices and critical-polynomial coefficients;
- Sylvester-resultant and common-divisor bridges;
- Cayley translation, Fourier projections, and numerator identities;
- large Dummit-kernel certificate families used by the computable decision route.

A reviewer should resist judging this layer by file count. The right question is whether each aggregate theorem imports all shards, whether the data semantics match the abstract polynomial object, and whether the final reduction advertises its use of `native_decide` or any implementation replacement.

The repository's root README identifies the Lean formalization and audit of Lazard's solvable-quintic algorithm as the most active ongoing work. The dedicated report is an audit-centered checkpoint, not a blanket claim that every printed assertion or every automatic solver endpoint is complete. Use the claim crosswalk, not the mere existence of a source module, to decide whether a particular formula is proved, corrected, refuted, locally checked, or still open.

4.8. Recommended algebra reading routes

4.8.1. For a theorem consumer

1. Start with [Algebra/README.md](#).
2. For the Jacobian result, read the local README and the public Lean/Rocq facades.
3. For polynomial formulas, read the first half of [Algebra/PolynomialFormulas/README.md](#), then the low-degree facade and examples.
4. Choose exactly one Abel–Ruffini theorem whose coefficient field and quantifiers match your use.
5. Treat the quintic and sextic decisions as separate APIs from the Lazard formula.

4.8.2. For a certificate auditor

1. Locate the aggregate theorem, not the generator.
2. Follow imports backward to the sparse representation and its semantics.
3. Search for `native_decide`, `implemented_by`, `unsafe`, and audit outputs.
4. Re-run the generator only after the checked data path is understood.
5. Compare Lean and Rocq at theorem granularity; the two ports do not have identical implementation strategies.

4.8.3. Focused builds

Representative commands are:

```
lake build JacobianConjecture
lake build +PolynomialFormulas
lake --dir Algebra/JacobianConjecture/Lean build

# The PolynomialFormulas tree is large. Prefer a facade or direct module
# while working on one certificate family.
lake env lean Algebra/PolynomialFormulas/Lean/PolynomialFormulas/Audit.lean
```

The precise audit module names vary by subproject. Use the local README and facade imports rather than assuming every generated shard is a standalone Lake target.

5. Analysis

5.1. Overview and an important documentation warning

The short [Analysis/README.md](#) no longer describes the scale of the directory. At the snapshot, analysis contains three very different projects:

Table 5.1.: Analysis project status.

Project	Status	Scope
Trigonometric identities	proved	Two compact Lean identities and independent Rocq counterparts: a golden-ratio sine sum and an eleven-term quadratic arctangent identity.
Exponential identities	partial	A proved three-base integer-exponent theorem, a tiny-tower floor certificate, and a very large formal research program around the open two-base Alaoglu–Erdős conjecture.
Fabius function	proved	Construction, uniqueness, exact dyadic evaluator, two complete paper formalizations, finite and global q-binomial formulas, corrected draft audits, and sharp small-argument asymptotics with periodic correction.

The public facades and project-local README files are therefore more reliable than the topic README. In particular, [Analysis/ExponentialIdentities/Lean/ExponentialIdentities.lean](#) now imports a large, actively developed theorem library that did not exist in the old walkthrough.

5.2. Trigonometric identities

5.2.1. Golden-ratio sine sum

Lean proves the exact identity

$$\sin 9^\circ + \sin 21^\circ + \sin 39^\circ = \frac{\varphi}{\sqrt{2}}.$$

The proof is intentionally elementary. The two off-center terms collapse by the sine sum-to-product formula, $\sin 9^\circ + \cos 9^\circ$ becomes $\sqrt{2} \cos 36^\circ$, and Mathlib’s exact value of $\cos(\pi/5)$ supplies the golden ratio. The theorem is in [Analysis/TrigonometricIdentities/Lean/TrigonometricIdentities/TrigGoldenRatio.lean](#).

5.2.2. Quadratic arctangent identity

The second module proves

$$\begin{aligned} &2939 \arctan(2)^2 - 1250 \arctan(3)^2 - 252 \arctan(4)^2 - 360 \arctan(5)^2 \\ &- 870 \arctan(7)^2 + 450 \arctan(8)^2 + 84 \arctan(13)^2 + 330 \arctan(18)^2 \\ &- 210 \arctan(21)^2 + 147 \arctan(38)^2 - 210 \arctan(47)^2 = 0. \end{aligned}$$

Rather than asking a nonlinear tactic to discover the cancellation, the proof linearizes all eleven arctangents in the lattice generated by

$$u = \frac{\pi}{4}, \quad a = \arctan \frac{1}{2}, \quad b = \arctan \frac{2}{3}, \quad c = \arctan \frac{1}{4}.$$

Addition, duplication, and reciprocal-complement lemmas derive each coordinate vector. The final Gram-matrix cancellation is then ring. This is a good small example of the repository's preferred style: expose the mathematical compression first, then let normalization verify the terminal identity.

5.3. Three integral powers force an integral exponent

Analysis/ExponentialIdentities/Lean/ExponentialIdentities/IntegerExponent.lean proves the unconditional theorem

$$2^x, 3^x, 5^x \in \mathbb{Z} \implies x \in \mathbb{Z} \quad (x \in \mathbb{R}).$$

The irrational case is ruled out by a square interpolation-determinant argument following Michel Waldschmidt's proof of the relevant special case of the Six Exponentials Theorem. The construction chooses a large finite six-dimensional box, forms the matrix

$$A_{ij} = \exp(a_i b_j),$$

proves its determinant nonzero from injectivity, proves every entry integral from the three power hypotheses, and obtains the lower bound $|\det A| \geq 1$. Analytic interpolation and explicit numerical inequalities give the contradictory upper bound $|\det A| < 1$.

Once x is rational, unique factorization at the base 2 shows its denominator is one. This second step is isolated as `integer_of_rational_of_two_pow_integer` and reused throughout the two-base project.

The underlying determinant machinery is split among `IntegerExponent.DeterminantBound`, `ExponentialKernel`, and `Grid`. A separate `GelfondSchneider` source subtree develops algebraic heights, denominators, and analytic bounds, but it is not imported by the public `ExponentialIdentities.lean` facade at this snapshot. Treat it as a separately reachable development unless a specific aggregate theorem imports it.

5.4. The tiny exponent tower

`Analysis/ExponentialIdentities/Lean/ExponentialIdentities/TinyExponentTower.lean` proves the exact floor of an expression whose visible scale is misleading:

$$\left| 10^{10^{10^{10^{10^{-10^{10}}}}} - 10^{10^{10}} \right| = 2,811,012,357,389.$$

The proof does not numerically evaluate the tower. It rewrites the tiny inner perturbation, derives rational upper and lower bounds for logarithms and exponentials, controls the propagated error, and traps the final difference between consecutive integers. This module is a compact model for exact transcendental bounding in Lean: the enormous ambient magnitude is irrelevant once the perturbation is normalized at the right layer.

5.5. The two-base Alaoglu–Erdős program

5.5.1. Statement and headline status

The classical conjecture is recorded, without being asserted, as

$$2^x, 3^x \in \mathbb{Z} \implies x \in \mathbb{Z}.$$

In Lean it is the proposition-valued definition `AlaogluErdosConjecture`. The conjecture remains **open**. The surrounding library is nevertheless one of the repository’s largest collections of unconditional reductions and obstructions. A correct walkthrough must therefore say both things at once:

The repository does not contain an unconditional proof of the Alaoglu–Erdős conjecture. It does contain a large kernel-checked theory of any hypothetical counterexample: irrationality, localization, exact finite exclusion through $2^x < 16384$, density-zero candidate bounds, radical and algebraic reformulations, determinant and valuation reductions, explicit no-go theorems, and several exact “all that remains is this transcendence/independence input” interfaces.

The public facade imports the entire active module chain. Consequently `lake build ExponentialIdentities` is no longer a small build.

5.5.2. Natural-candidate reduction

If $2^x = m \in \mathbb{N}$, then

$$3^x = m^\alpha, \quad \alpha = \frac{\log 3}{\log 2}.$$

The problem becomes the classification of natural m for which m^α is also integral. The base file proves:

- a nonintegral exponent with integral 2^x is irrational;
- natural multiples of a solution are again solutions at each base;
- an integral power strictly between consecutive powers stays at least one integer unit from either endpoint;
- normalized inequalities force the fractional part of a hypothetical exponent away from 0 and 1;
- the second difference of $m \mapsto m^\alpha$ is strictly between 0 and 1 in the relevant range, so three consecutive natural candidates cannot all succeed.

The definitions `TwoBaseNaturalCandidate`, the logarithmic conversion lemmas, and the integer-gap estimates are the common front end used by later analytic and arithmetic modules.

5.5.3. Exact finite exclusion through $x < 14$

The modules `FiniteCheck`, `FiniteCheck256`, `FiniteCheck4096`, `FiniteCheck8192`, and `FiniteCheck16384` build an exact certificate ladder. The strongest endpoint proves:

$$2^x, 3^x \in \mathbb{Z}, 2^x < 16384 \implies x \in \mathbb{Z}.$$

Thus every nonintegral solution would satisfy $x \geq 14$.

For each non-power-of-two integer $m < 16384$, the committed table records $a = \lfloor m^\alpha \rfloor$ and a tier selecting rational lower and upper enclosures for α . Exact big-integer comparisons prove

$$a^q < m^p \quad \text{and} \quad m^{p'} < (a+1)^{q'},$$

for suitable rational bounds $p/q < \alpha < p'/q'$. Lean replays every row with kernel `decide`; the table-generation scripts and `mpmath` calculations are not the soundness boundary.

This certificate architecture scales poorly in file size but well in trust clarity: every numerical row becomes a finite integer inequality, and the only external role is to discover compact witnesses.

5.5.4. Four Exponentials reduction

The module `FourExponentialsReduction` proves two rational linear-independence facts unconditionally:

$$\{1, x\} \text{ is independent over } \mathbb{Q} \quad \text{when } x \notin \mathbb{Q},$$

and

$$\{\log 2, \log 3\} \text{ is independent over } \mathbb{Q}.$$

It then proves that the real specialization of the Four Exponentials Conjecture implies `AlaogluErdosConjecture`. The conjectural input is itself a `Prop`-valued definition, not an axiom. This is the cleanest formal explanation of the classical relationship: for an irrational counterexample, all four exponentials in the 2×2 grid would be algebraic, contradicting Four Exponentials.

5.5.5. Radical and algebraic reformulations

A large cluster of modules studies what happens after adjoining rational powers and canonical radical fields. The public import list includes:

- `RadicalReformulation`, `PrimitiveRadical`, and `RadicalDegree`;
- `CanonicalRadicalFieldNorm`, `CanonicalRadicalValuation`, and `CanonicalRadicalSharpness`;
- denominator normalization at the prime 3 and rational power-index lemmas;
- primitive-generator and simultaneous-radical-degree constructions;
- rational and algebraic third-base/six-exponentials bridges;
- algebraic output loci, divisible hulls, mixed outputs, and rank-three equivalences.

These files are best read as a network of exact equivalences and consequences, not as independent attempted proofs of the conjecture. They identify where algebraicity, radical degree, field norm, and prime valuations would have to become rigid enough to force the control pair $(2^n, 3^n)$.

5.5.6. Counting, equidistribution, and density

The sequence m^α is convex with decreasing curvature. The library exploits that geometry at several scales:

- `ExactCounting` and `QuadraticAsymptotic` count candidate patterns and control second-order terms;
- `BlockWeyl` and `BlockEquidistribution` package finite/block equidistribution statements;
- `FractionalPartDensity`, `NonintegerDensity`, and `ShrinkingTarget` formulate the relevant distribution and shrinking-target interfaces;
- `ZeroDensity` combines a proved arithmetic-progression curvature obstruction with Mathlib’s formalized Roth theorem.

The last result is unconditional: the number of positive natural candidates below N is $o(N)$, so their proportion tends to zero. The proof strengthens the consecutive-triple obstruction to arbitrary three-term progressions of step h when $h^5 \leq n$, then invokes Roth. Zero density is much weaker than finiteness, but it is a genuine global theorem about the candidate set.

5.5.7. Higher differences and exponential-polynomial rigidity

The modules `ThirdDifference`, `HigherDifference`, `PolynomialApproximationTax`, `FallingFactorialBound`, and `ExponentialPolynomialZeros` formalize increasingly high-order finite-difference control. Their role is to quantify the cost of a long block of near-integral or exactly integral values of a nonintegral power function.

A parallel determinant route develops arbitrary-node alternants, semigroup determinants, Vandermonde and Smith profiles, resultants, divided differences, moment arithmetic, and rank geometry. Representative modules include:

- `ArbitraryNodeDeterminant`, `SemigroupDeterminant`, and `SuperfactorialDeterminant`;
- `GeometricVandermondeSmith`, `SmithProfileCore`, and `ContentNormalizedProfileFinite`;
- `ClearedDividedDifference`, `CrossResultantBridge`, and `DeepSubresultantCost`;
- `CandidateAlternant`, `FormalPrimeDeterminant`, and `FermatQuotientDeterminant`.

The common theme is to convert many exact power values into a determinant that is simultaneously arithmetically divisible and analytically small. Several modules prove the algebra downstream of a proposed lower bound while leaving that lower bound as the explicit frontier.

5.5.8. Valuations, tropical blocks, and finite-place relations

Another branch represents candidates by prime-exponent vectors and studies the induced min-plus geometry. The facade imports modules for valuation lattices, tropical initial forms, block fiber factorization, finite-place short relations, unit ordering gains, row-block Laplace expansion, assignment-excess geometry, and global prefix cascades.

This branch is unusually honest about failed compression steps. Files such as `DenominatorCancellationCount` `erexample` and `GridTransformNoGo` prove that tempting cancellations or transforms do not deliver the required gain. Such counterexamples are part of the research result: they prevent future work from silently reintroducing a false lemma in a more elaborate notation.

The current head also exports `SmoothLevelClassification`, the kernel-decided finite core of the prime-necklace transform. If

$$R_h = \frac{3^h - 1}{2^h - 1}$$

is reduced to numerator and denominator, then for $1 \leq h \leq 12$ both parts are $\{2, 3\}$ -smooth exactly when $h \in \{1, 2, 4\}$. The exact exceptional ratios are $R_1 = 2$, $R_2 = 8/3$, and $R_4 = 16/3$, and a companion theorem excludes every level $5 \leq h \leq 12$. The implementation deliberately uses a fuel-bounded stripping recursion that ordinary

kernel `decide` can evaluate. It does *not* claim the infinite classification: excluding every $h > 12$ still requires a primitive-divisor input of Zsigmondy type.

5.5.9. Orbit, carry, Kummer, and interpolation branches

The later import surface explores several further mechanisms:

- compact-orbit and adelic-drift formulations;
- Sturmian probability and generating-series models for rounding patterns;
- matched-gap amplification and logarithmic-product independence;
- factorial cocycles, mixed-radix carries, borrow blocks, and Kummer compatibility;
- Newton interpolation, reciprocal moment lattices, Fejér-kernel certificates, and prime-top isolation;
- toric monomial rigidity, consecutive-dilation rigidity, and Abel-slice mass.

The point of the facade is not that all these routes independently prove the conjecture. It is that their proved lemmas, exact failure modes, and conditional endpoints coexist in one elaborated dependency graph. The 1.2-megabyte unified report under [Analysis/ExponentialIdentities/Article/alaoglu-erdos-unified-report](#) supplies the human research narrative; the Lean modules supply the theorem-level ledger.

5.5.10. The weight-two collapse frontier

The final imported module at the snapshot, `WeightTwoCollapse`, isolates a particularly crisp open input. Expand

$$\log 2 \log A = \log 3 \log M$$

in the prime basis. Unique factorization gives divisor vectors m_p, a_p , and the relation yields coefficients of products $\log 2 \log p$ and $\log 3 \log p$. The module proves that the coefficient array is exactly the previously defined two-star control tensor.

If those weight-two logarithmic products were linearly independent over $\mathbb{Q}$, every coefficient would vanish. The proved theorem `collapse_to_control` then forces

$$M = 2^t, \quad A = 3^t$$

at the divisor-vector level. The transcendence/independence premise is open; all algebra after coefficient vanishing is finished. This is a model conditional endpoint: the missing statement is narrow, named, mathematically substantive, and not hidden in a generic “regularity” hypothesis.

Do not read the facade top-to-bottom as a linear proof attempt. Start with `TwoBaseIntegerExponent.lean`, then choose one route: finite certificates, density, radical fields, determinants, valuations/tropical blocks, or the weight-two collapse. For each route, identify the unconditional theorem, the conditional bridge, and any explicit no-go module. Use the unified report only after the formal dependency boundary is clear.

5.6. The Fabius function

5.6.1. Public scope

The current `Analysis/FabiusFunction/Lean/FabiusFunction.lean` facade imports the bounded Fabius function, its signed global extension, exact dyadic arithmetic, complete formalizations of two arXiv papers, audits of two local drafts, the finite and global q-binomial identities, and the corrected sharp small-argument asymptotic. The local README explicitly reports no sorry in the project.

This is a major change from the old walkthrough, which treated Fabius as a statement formalization. It is now a completed analytic development with executable exact arithmetic and a substantial asymptotic theory.

5.6.2. Construction and uniqueness

The bounded function is characterized in a CDF-style formulation by a fixed-point integral equation. The development builds the relevant contraction operator, proves existence and uniqueness, and derives the expected functional and regularity properties. A signed extension `extendedFabius` makes the global differential/dilation equations convenient without losing the canonical bounded object.

The project separates three layers that are often blurred in informal accounts:

1. the analytic function and its uniqueness;
2. exact rational recurrences for moments and dyadic values;
3. source-paper theorems and asymptotic consequences.

This separation permits the dyadic evaluator to be executable while its equality with the analytic function remains a theorem.

5.6.3. Exact dyadic evaluator

The functions `fabiusDyadicValue` and `evalFabiusDyadic` compute exact rational values at dyadic arguments. Their recurrences are derived from the analytic equations and moment identities, and the correctness theorem applies at every dyadic point, not only to a finite table.

The important implementation property is that evaluation returns rational data. No floating-point approximation, oracle for a real integral, or unproved lookup table is needed at runtime. The analytic development proves that the computed rational is the actual Fabius value.

5.6.4. Complete paper formalizations

The facade imports `Paper05442` and `Paper06487`, corresponding to arXiv:1702.05442 and arXiv:1702.06487v3. Their source claims are not merely catalogued: the README describes the formalizations as complete and proved. Exact dyadic arithmetic, moment identities, and distributional representations provide the common substrate.

The project also imports `PaperFabiusAsymptotic` and `PaperKFoldThueMorse`. These are claim-level audits of local TeX drafts. False claims are represented by formal counterexamples or corrected statements rather than silently repaired in prose. This makes the assistant development an errata ledger as well as a proof.

5.6.5. Finite q-binomial identities and the global binary-reduction series

The asymptotic aggregate now includes a proof of the finite Thue–Morse/q-binomial formula in its full dyadic scope. For every natural m, n , including unreduced representations and values outside the unit interval, the displayed rational sum computes the signed global Fabius value at $m/2^n$. A separate bounded corollary applies under $m \leq 2^n$. The inner polynomial is proved invariant under arbitrary common rational translation, so the source’s half-shifted form, the centered form, and the raw-coordinate specialization are connected by theorems rather than by informal algebra.

The new modules `FabiusBinaryReductionSeries`, `FabiusQBinomialTaylor`, and `FabiusGlobalQBinomialSeries` then restore the missing scale-zero term in the proposed infinite formula. For every real $x \geq 0$, the resulting series converges absolutely to the signed global extension; on $[0, 1]$ it specializes to the bounded function. Polynomial constancy upgrades the translation parameter from rational to arbitrary real or complex values. The endpoint discrepancy of the former one-indexed formula is also isolated exactly: the omitted term vanishes on $0 \leq x < 1$ but equals one at $x = 1$.

5.6.6. Negative Laplace product and periodic correction

The asymptotic side begins with an exact negative-Laplace product. Mellin analysis then exposes a Gamma–zeta periodic correction. The key conceptual correction is that the sharp small-argument asymptotic does not have a single constant prefactor: a nonconstant periodic function of the logarithmic scale survives.

The public facade imports a sequence of modules that make this route inspectable:

- `NegativeLaplace`;
- `PeriodicCorrection`;
- `MellinBose` and `MellinFinitePart`;
- `BoseFinitePartIntegral`;
- `PeriodicMean` and `PeriodicRegularity`;
- `LaplacePeriodicSecondOrder`.

Together they establish the unconditional corrected sharp asymptotic and second-order periodic behavior. The finite-part and Bose-integral modules isolate delicate analytic continuation and regularization steps instead of burying them in the endpoint proof.

5.6.7. Trust and source status

The formal theorem boundary is Lean/Mathlib. Numerical experiments and draft TeX files motivate formulas but are not imported as proof. The project README records paper errata and exact module roles; the facade is the best reachability check.

A focused build is simply:

```
lake build FabiusFunction
```

For a narrower audit, elaborate the specific paper or asymptotic facade and inspect its audit output. Because the project mixes executable dyadic computation with noncomputable analysis, a reviewer should distinguish reduction of rational recurrences from proofs about integrals, transforms, and asymptotic filters.

5.7. Analysis reading order

A practical order is:

1. read the two small trigonometric modules for local proof style;
2. read `IntegerExponent.lean` for the finished three-base theorem;
3. read `TinyExponentTower.lean` for exact transcendental bounding;
4. treat the two-base tree as a research program and select one branch rather than loading the whole conceptual graph at once;
5. read the Fabius README, facade, construction, dyadic evaluator, q-binomial series modules, and then the paper/asymptotic endpoints.

6. Combinatorics

6.1. Overview

The combinatorics tree is now one of the repository’s most varied subject areas. It contains conventional finite proofs, exact real-algebraic value certificates, ordinal normal-form computations, geometric tiling certificates, an open coefficient-support conjecture, and paper-scale Ramsey formalizations. The topic README remains useful for the older projects, but it predates the current [Combinatorics/Ramsey](#) tree; the directory facades and project-local READMEs are therefore authoritative for this chapter.

Project	Status	Principal content
A290268	partial	Derivative coefficient recurrence, closed form and generating function, exact support reduction; the decisive nonvanishing inclusion remains open.
PowerTowers	mixed	Shared expression syntax; exact initial-value proofs for natural, ordinal, and principal-complex towers; a carefully separated conditional/data frontier for A198683 at size 12.
A158415	proved	Exact real-expression semantics and a proof that the fifteenth value is 791, with explicit order and range certificates.
KlarnerConstant	proved	Exact geometric transfer inequalities and the rational upper bound $\lambda \leq 9047/2000 = 4.5235$.
SquaredSquare	partial	Independent Lean/Rocq verification of Duijvestijn’s order-21 perfect squared square and the lower bound seven; sharp minimality is reduced to the exhaustive-search claim.
Ramsey papers	mixed	Statement catalogues, common definitions, proof layers, exact finite counts, and an expanding Gowers–Szemerédi development.

6.2. Derivative expansions and OEIS A290268

The development under

[Combinatorics/DerivativeExpansions/A290268/Lean](#)

formalizes the coefficient array arising when differentiating x^{x^2} repeatedly and fully expanding. The sequence $a(n)$ counts nonzero terms. The facade [Combinatorics/DerivativeExpansions/A290268/Lean/A290268.lean](#) imports six layers:

1. Core: the coefficient recurrence and basic support restrictions;
2. ClosedForm: the quasi-polynomial, linear recurrence, and generating-function presentations, with equivalence proofs;
3. Count: the conjectured support set S_n and its cardinality;
4. Structural: vanishing and geometric support constraints;
5. Table: checked finite values; and
6. Main: the exact reduction of the sequence conjecture to support equality.

The most useful endpoint is not a falsely advertised proof of the OEIS conjecture. It is the theorem `a_eq_closedForm_of_support`: if

$$\text{coeff}(n, j, k) \neq 0 \iff (j, k) \in S_n,$$

then $a(n)$ equals the proved closed form. A second endpoint, `a_eq_closedForm_of_two_sided`, separates the two inclusions.

The structural direction

$$\text{coeff}(n, j, k) \neq 0 \implies (j, k) \in S_n$$

is supported by parity, triangular-region, and hole-line vanishing theorems. The remaining mathematical core is the converse nonvanishing statement on the conjectured support. This is a good example of the repository's preferred treatment of an open OEIS claim: formalize the definitions, prove the surrounding recurrence and enumeration theory, and isolate the open assertion as one reusable hypothesis rather than treating a finite table as a proof of the general formula.

A290268 claim boundary

The facade proves equivalence and reduction theorems around the conjecture. It does not prove the universal support-nonvanishing assertion. A reviewer should cite the conditional endpoint together with the exact residual hypothesis, not simply say that “A290268 is formalized.”

The older [0eis/A290268](#) path is retained as a historical or redirecting surface. New work belongs under the subject tree.

6.3. A shared framework for power towers

PowerTowers uses a common lexical datatype of fully parenthesized binary expressions. A leaf is one token; an internal node applies a chosen exponentiation-like operation. This separates three questions that are often conflated in sequence programs:

1. which expression trees are syntactically legal;
2. what semantic value each tree denotes; and
3. which computational normal form can compare those values exactly.

The root facade [Combinatorics/ExpressionEnumeration/PowerTowers/Lean/PowerTowers.lean](#) imports four principal sequences and a substantial A198683 certificate tree.

6.3.1. A000081: the syntax baseline

A000081, the number of rooted unlabeled trees, supplies a familiar combinatorial baseline and reusable recurrence machinery. In this repository its role is architectural as much as enumerative: it demonstrates how a canonical recursive object family is related to the common expression-size infrastructure.

6.3.2. A002845: towers of twos

A002845 counts distinct natural values of all parenthesizations of

$$2^{2^{\cdot^{\cdot^{\cdot}}}}$$

with a fixed number of literal twos. Direct natural evaluation becomes absurdly large almost immediately, so the development passes to the exact base-two logarithm. Every semantic value is 2^m , and injectivity of $m \mapsto 2^m$ proves equality of the semantic and logarithmic counts.

The computation-facing representation is a canonical sparse binary integer. The verified operation `Sparse.shift` implements

$$(a, b) \mapsto a 2^b$$

on exact logarithms. The module proves canonicity and semantic correctness of every combine, then connects the dynamic-programming sparse levels to the lexical definition. There is no `implemented_by` substitution in this path: the efficient sparse operation is itself the proved operation. Large tabulated values are discharged with `native_decide`, so the remaining computational trust is the normal native-decision boundary rather than an unrelated external program.

6.3.3. A199812: towers of ordinals

A199812 replaces the atom by ω and interprets a node by ordinal exponentiation. Every result is a principal power of ω . The executable representation stores the exponent as a Mathlib `ONote` in Cantor normal form below ϵ_0 .

The central bridge proves that the normal-form exponent computation denotes exactly the lexical ordinal expression. Injectivity of ordinal exponentiation at base $\omega > 1$, together with injectivity of normal-form representation, transports cardinalities between semantic ordinals and finite sets of notes. The project thereby avoids pretending that arbitrary ordinal equality is a black-box executable operation.

6.3.4. A198683: principal powers of i

A198683 counts distinct principal values of all parenthesizations of $i^i \dots^i$. Unlike the natural and ordinal cases, principal complex exponentiation is neither associative nor globally injective, and equality questions combine branch cuts, periodicity, transcendental functions, and rapidly nested magnitudes.

The Lean development unconditionally proves the values through size seven; in particular `a198683_seven` states $a(7) = 34$. The repository also contains accepted external data through $n = 11$, exact Schoenfeld-row certificates, analytic separation lemmas, and a large size-12 investigation. The local research ledger is deliberately more precise than a single sequence line:

- accepted sequence data and exploratory computations are evidence, not kernel theorems;
- conditional partition arguments narrow the size-12 count;
- a proved near-one separation removes one ambiguity;
- the remaining overflow “no miracles” obligation is the barrier between the final candidates; and
- without the required partition witness, later values are not promoted to unconditional theorems merely because a numerical program printed them.

The detailed status is maintained in [Combinatorics/ExpressionEnumeration/PowerTowers/Research/A198683/README.md](#). For this project, the research README is part of the formal interface: it tells a reviewer which Lean files prove unconditional facts, which theorems consume hypotheses, which rows are data certificates, and which observations are heuristic.

For A002845 and A199812, inspect the semantic-to-normal-form bridge before trusting a computed count. For A198683, additionally inspect the branch convention and the status ledger. A list of decimal approximations is never an equality certificate for principal complex values.

6.4. Radical expressions and OEIS A158415

OEIS A158415 counts distinct real values of expressions generated by the nullary symbol 1, unary square root, and binary addition, with size equal to the total number of symbols. The project lives at [Combinatorics/ExpressionEnumeration/RadicalExpressions/A158415](#).

6.4.1. Exact lexical semantics

`Combinatorics/ExpressionEnumeration/RadicalExpressions/A158415/Lean/A158415/Core.lean` defines the expression syntax, lexical enumeration, real evaluation, value sets, and a split-recursive presentation. It proves the two presentations equal. This is the key replacement for the historical OEIS program's floating comparison with an epsilon: equality is proved in the ordered real field using exact radical identities and inequalities.

A padding operation replaces a leaf 1 by $\sqrt{1}$, increasing size without changing value. This yields monotonicity of the represented value sets and is repeatedly used to control recursive branches.

6.4.2. The size-15 certificate

The facade `Combinatorics/ExpressionEnumeration/RadicalExpressions/A158415/Lean/A158415.lean` imports the core and the size-15 endpoint. The certificate is split into generated value tables, exact order proofs, and range-inclusion modules. The result file proves

$$a158415(15) = 791.$$

The two hard directions are explicit:

1. every recursively generated value of size 15 occurs in the enumerated table; and
2. every table entry is denoted by a legal size-15 expression.

Strict ordering of the table gives distinctness and hence cardinality 791. The proof does not depend on a decimal tolerance. Support scripts may generate or organize the table, but Lean rechecks the exact radical equalities, inequalities, range membership, and cardinality.

The parallel Rocq tree is independently structured rather than generated from Lean source. When parity matters, compare theorem statements and semantic conventions, not source syntax.

6.5. Klarner's polyomino growth constant

The Klarner-constant project formalizes a transfer-matrix style upper-bound argument for the exponential growth of fixed polyominoes. Its main current numerical endpoint is

$$\lambda \leq \frac{9047}{2000} = 4.5235.$$

The development is notable for how it handles certificate discovery. Python and Wolfram-language programs search for rational weights and recurrence certificates. The committed Lean objects are exact finite lists and rational inequalities. The theorem endpoint consumes a `GeometricComplete` package tying the finite geometric states, child recurrences, weights, and global counting argument together. Ordinary kernel reduction and `norm_num` recheck the certificate; the README explicitly reports no `native_decide`, `unsafe`, or admitted theorem in the checked path.

The right audit order is:

1. read the definition of the geometric states;
2. check that the finite child relation is complete, not merely sound for generated children;
3. inspect the rational weight inequalities;
4. follow the induction from local transfer bounds to the global growth estimate; and
5. only then inspect the support generator that found the certificate.

6.6. Perfect squared squares

SquaredSquare contains independent Lean and Rocq proofs that every positive-side square admits a perfect dissection into 21 pairwise noncongruent squares. The concrete certificate is Duijvestijn's order-21 dissection of the side-112 square, with tile sides

$$2, 4, 6, 7, 8, 9, 11, 15, 16, 17, 18, 19, 24, 25, 27, 29, 33, 35, 37, 42, 50.$$

The formal geometry uses axis-parallel closed squares in $\mathbb{R}^2$. A dissection requires positive side lengths, pairwise disjoint open interiors, and exact coverage. Congruence is not reduced by definition to equality of side lengths: the development proves, using squared diameter, that congruent nondegenerate squares have equal sides, while translation supplies the converse.

The integer certificate verifies placement, separation, distinct side lengths, and coverage of every unit grid cell in $[0, 112]^2$. Lean checks this with kernel `decide +kernel`; Rocq uses `vm_compute`. Floor and integer-part lemmas lift the finite grid certificate to exact real-plane coverage. Scaling then gives a 21-piece perfect dissection of every square with positive side.

The elementary minimality development proves that every perfect squared square has at least seven pieces. It formalizes the corner-tile argument, one-dimensional edge partitions, and the contradictions for four, five, and six pieces. Thus both provers establish

$$7 \leq \text{minimum order} \leq 21.$$

The sharp claim that 21 is minimal is not smuggled in as an axiom. `MinimalOrder` defines `DuijvestijnSearchClaim`, the missing exhaustive-search theorem ruling out orders 7 through 20, and proves the desired least-order statement equivalent to it. A future complete proof would require a verified-complete enumeration of the relevant planar networks or placement orders and certified linear algebra for every case.

6.7. Ramsey theory and progression-avoiding permutations

The current `Combinatorics/Ramsey` tree is a major addition since the previous walkthrough. It combines paper transcriptions, shared formal definitions, exact finite enumeration, and proof modules for several papers on arithmetic progressions in permutations.

6.7.1. Common formal vocabulary

`Combinatorics/Ramsey/Lean/RamseyPaperCommon.lean` provides bridges among the conventions used by the papers: finite permutations, displayed words, monotone arithmetic progressions, integer-signed progressions, avoidance predicates, and finite/infinite formulations. The common layer prevents each paper port from quietly choosing a different indexing or monotonicity convention.

For three-term progressions, a word with distinct entries avoids both increasing and decreasing arithmetic progressions exactly when no middle-position value is the arithmetic mean of two surrounding-position values. The executable predicate `MidpointFree` is proved equivalent to the paper definitions before it is used for counting.

6.7.2. Paper catalogues and proof layers

The current top-level facades include:

- `Combinatorics/Ramsey/Lean/DavisEntringerGrahamSimmons1977.lean`;
- `Combinatorics/Ramsey/Lean/Sharma2012.lean`;
- `Combinatorics/Ramsey/Lean/LeSaulnierVijay2010.lean`;
- `Combinatorics/Ramsey/Lean/LeSaulnierVijay2011.lean`; and
- `Combinatorics/Ramsey/Lean/GowersSzemerédi.lean`.

The first, Sharma, and the 2011 LeSaulnier–Vijay ports have complete statement catalogues with corresponding proof imports. The 2010 facade is intentionally more conservative: it exposes the catalogue, while a separate `LeSaulnierVijay2010/Proofs.lean` supplies many `..._holds` theorems through shared and later-paper results. A user importing only the statement facade should not infer that every numbered assertion is proved.

The Gowers facade catalogues the numbered claims in *A new proof of Szemerédi’s theorem* across sections 1–18. Each source claim has a proposition-valued definition; completed proofs are exported as separately named `..._holds` theorems. The proof layer is large and growing, but the naming convention deliberately makes missing endpoints visible.

At the surveyed head, the facade additionally reaches `Proofs09Restriction`: a completed random-restriction proof of Lemma 9.3 and Corollary 9.4, preceded by a formal counterexample showing why the conventional hypothesis $\eta \leq 1$ is necessary. The Section 15 restriction layer was expanded in the same merge series. These additions exemplify the project’s preferred workflow: repair the exact source statement first, then export its `_holds` endpoint through the public facade.

The `Combinatorics/Ramsey/Papers` directory contains paper sources, scans, and corrected transcriptions. These are provenance and review aids. The Lean statements, not the OCR text, determine what has been formalized.

6.7.3. Exact finite counting through 20

The shared counter defines `reflectedM n` logically as the cardinality of all permutations of `Fin n` satisfying the decidable midpoint test. The certificate theorem records the exact values

n	1	2	3	4	5	6	7	8	9	10
$M(n)$	1	2	4	10	20	48	104	282	496	1066

and

n	11	12	13	14	15	16	17	18	19	20
$M(n)$	2460	6128	12840	29380	74904	212728	368016	659296	1371056	2937136

The source-level definition is a direct finite enumeration. Its executable implementation is replaced with `fastReflectedM`: a reviewed prefix-extension recurrence, using a small C implementation for $n \leq 20$ and a Lean fallback outside that range. The theorem `Combinatorics/Ramsey/Lean/RamseyPaperCommon/CountCertificates.lean` is discharged by `native_decide`.

This has a wider trust boundary than kernel reduction alone. The theorem statement and specification bridges are Lean theorems, but execution passes through Lean’s native compiler, `implemented_by`, the FFI declaration, and `Combinatorics/Ramsey/Lean/RamseyPaperCommon/ThreeFreeCounter.c`. The tiny `RamseyCounterShim` exists so the certificate target can link the C object without compiling Mathlib’s entire dependency closure into a shared library.

What the native table certifies

The table is a theorem produced by native decision against the logical `reflectedM` definition, whose executable realization is the reviewed extension-equivalent counter. It is not a pure-kernel normalization of all $n!$ permutations, and it is not an unaudited CSV import. Reproducibility therefore requires building the dedicated native certificate target, not merely elaborating the statement module.

6.8. Combinatorics reading routes

For exact-computation architecture, read `A002845`, `A158415`, `SquaredSquare`, and then the Ramsey counter; these four projects deliberately use four different verification boundaries. For an open-problem reduction, read `A290268` and the `A198683` ledger. For paper formalization, begin with `RamseyPaperCommon`, then compare one statement facade with its proof facade before entering Gowers–Szemerédi. For geometric certificates, read `Klärner` and `SquaredSquare` side by side: one certifies inequalities over a finite state space, the other certifies an exact placement and coverage.

7. Computability

7.1. Overview

The computability tree contains three projects with very different proof styles:

1. quotient and oracle-machine theory for Turing degrees;
2. syntactic compilers and operational simulations for combinatory logic; and
3. finite machine classification and domination arguments for Busy Beaver functions.

The common theme is explicit semantic scope. The projects avoid saying “Turing complete” or “same degree” until the operational model, reduction notion, divergence behavior, and any effective-enumeration assumptions have been named.

7.2. Turing degrees

TuringDegrees formalizes Turing reducibility for sets of natural numbers and organizes its coverage against a pinned revision of the reference article. The principal documentation artifact is [Computability/TuringDegrees/COVERAGE.md](#); it records theorem-by-theorem Lean/Rocq parity and every logical or effective-enumeration premise.

7.2.1. Lean core

The Lean development includes:

- characteristic-oracle reducibility and Turing equivalence;
- quotient degrees, a partial order, and the least degree $\mathbf{0}$;
- exact even/odd set join and the induced join-semilattice;
- complement equivalence;
- the halting degree $\mathbf{0}'$ and $\mathbf{0} < \mathbf{0}'$;
- cardinality of an individual degree class, of the entire degree space, lower cones, and strict upper cones;
- c.e. degrees, many-one completeness of the halting set, and the fact that every c.e. degree lies below $\mathbf{0}'$; and
- order-theoretic consequences of hypothetical minimal degrees, missing greatest lower bounds, and exact pairs.

Several of these results exploit Mathlib infrastructure that has no direct Rocq analogue, particularly quotient cardinalities and arbitrary-set cardinal arithmetic.

7.2.2. Rocq extensions and explicit hypotheses

The Rocq side contains a more developed jump and hierarchy theory. It proves jump invariance and monotonicity, strictness above every degree, finite iterates, versions of Post’s theorem, the Shoenfield limit lemma, and a Friedberg–Muchnik/Post-problem result. Where such theorems depend on an effective partial-function enumeration, finite-string coding, Markov’s principle, restricted excluded middle, or a constructive Post-problem principle, those ingredients remain theorem parameters.

This is not merely foundational politeness. Removing an effective enumeration premise can change a theorem from a computability result into an unprovable choice principle. The coverage ledger therefore distinguishes an unconditional pure-order consequence from an existence theorem imported through a concrete computability library.

7.2.3. Deliberate nonclaims

The project does not postulate deep results solely to make the checklist look complete. Among the explicitly absent results are existence of minimal degrees, density of the c.e. degrees, jump inversion, arbitrary continuum antichains, nonlattice witnesses, exact-pair existence, and broad first-order-theory classifications. The ledger also corrects two common informal formulations: recursive approximations need stage and input arguments, and the useful exact-pair theorem has non-strict common lower bounds plus separate strict domination of the sequence.

Cite the theorem and its prover. A statement may be unconditional in Lean, conditional in Rocq, present only as an order-theoretic implication, or absent as an existence theorem. “The repository formalizes Turing degrees” is true but too coarse for any technical dependency.

The principal Lean checks are:

```
lake build +TuringDegrees +TuringDegrees.Audit
```

The Rocq build is part of the root project and may require the pinned `coq-synthetic-computability` submodule.

7.3. Combinatory logic: SK, SKI, and Iota

This project proves, independently in Lean and Rocq, the standard universality chain from closed weak untyped lambda calculus through pure SK and SKI to the one-combinator Iota language. The development is substantially stronger than checking the folklore identities for I , K , and S .

7.3.1. The source calculus and bracket abstraction

The lambda syntax is intrinsically scoped. Substitution is capture-avoiding by construction. The operational relation is weak contextual beta reduction: reductions are closed under applications but not under lambda bodies. An occurs-aware bracket-abstraction compiler maps closed lambda terms to pure SK.

The key simulation theorem is positive: a beta contraction maps to one or more SK contractions, not merely to a reflexive-transitive path that could be empty. Application is preserved exactly, and the simulation lifts to both positive and reflexive-transitive closures. Size bounds make the compiler quantitatively inspectable.

7.3.2. SK, SKI, and the noninjective reverse compiler

Pure SK embeds constructor-for-constructor into SKI. The conventional SKI-to-SK compiler replaces primitive I by SKK . It cannot be injective because primitive I and the literal SKI term SKK collide. Both provers formalize this collision rather than claiming a nonexistent isomorphism.

7.3.3. Pure Iota and its runtime

The Iota source language has one leaf ι and binary application. Runtime terms additionally contain auxiliary S and K nodes, with rules

$$\iota x \rightarrow xSK, \quad Kxy \rightarrow x, \quad Sxyz \rightarrow xz(yz).$$

Keeping auxiliaries out of source syntax prevents a supposed Iota compiler from hiding extra primitives in its output.

The homomorphic SKI-to-Iota compiler uses

$$I \mapsto u, \quad K \mapsto \iota(\iota(u)), \quad S \mapsto \iota(\iota(\iota(u))).$$

The development proves injectivity, a linear size bound, positive simulation of every SKI contraction, and a checked nonempty reduction cycle for compiled omega.

7.3.4. Embedding Iota back into lambda terms

A separate translation interprets the complete runtime by the usual closed lambda terms for S , K , and $\iota = \lambda f.fSK$. Exact one-, two-, and three-beta-step head traces establish the runtime rules. Context lifting, closure preservation, and syntactic injectivity produce a faithful application-homomorphic operational embedding of pure Iota into closed weak lambda calculus.

The result is an expressiveness loop, but not a full abstraction theorem. The compilers are not claimed to be mutual syntactic inverses or reduction-reflecting in both directions.

7.3.5. Partial recursion and Turing machines

The Rocq project additionally connects a strong lambda calculus, partial recursive relations, and Turing machines through the focused vendored snapshot of the Coq Library of Undecidability Proofs. It proves arity-indexed equivalences of partial functional relations, preserving undefinedness rather than only total outputs.

That upstream library uses its own unscoped weak-call-by-value calculus and Scott numerals. The README explicitly declines to identify it with the project’s intrinsically scoped contextual calculus. Composing the exact Turing-machine equivalence through the local Iota compiler is therefore a remaining semantic-bridge obligation, not an automatic corollary of two unrelated simulations.

7.3.6. Audits

The direct SK/SKI/Iota runtime results are particularly clean. Lean’s axiom report for the broad project exposes only standard principles used by surrounding extensionality and quotient infrastructure; the core runtime simulations and concrete cycles are axiom-free. Rocq’s Print Assumptions and coqchk audits report no project-specific axioms.

A focused Lean build is:

```
LEAN_NUM_THREADS=0 lake build +CombinatoryLogic
```

The exact recursive-function and Turing-machine equivalences require the committed vendored closure and the Rocq/MetaRocq version documented by the project README.

7.4. Busy Beaver

The Busy Beaver tree defines the repository’s own small Turing-machine model, score and time measures, domination theorems, finite classifications, and bridges to the vendored BB5 certificate corpus.

7.4.1. Score and time are different functions

The score function $\Sigma(n)$ maximizes the number of nonblank symbols left by a halting n -state machine. The time function $BB(n)$ maximizes the number of steps. The formalization keeps them separate and proves the classical small values

$$\Sigma(2) = 4, \quad \Sigma(3) = 6, \quad \Sigma(4) = 13,$$

and

$$BB(2) = 6, \quad BB(3) = 21, \quad BB(4) = 107.$$

The differing sequences are intentional, not a discrepancy between provers.

7.4.2. Finite classification targets

The Lean BB2 and BB3 proofs are large opt-in targets. They normalize machine symmetries, execute bounded traces, and discharge exhaustive classifications. They are not placed in the root default target because their cost is disproportionate to routine integration builds.

For BB4, Lean proves soundness of a transition-normal-form reduction. The final equality between the reduced root enumeration and all relevant machines remains conditional on an exhaustive-root premise. The vendored Rocq certificate development supplies independently checked time certificates and bridges, but the local documentation keeps the provenance and measure conventions visible.

Useful commands are:

```
lake build +BusyBeaver.BB2
lake build +BusyBeaver.BB3
lake build +BusyBeaver.Mathlib
```

Heavy classifications should be run serially under the repository's worker policy. A successful facade build is not evidence that every optional classification target has executed.

7.4.3. Domination results

Beyond small values, the project proves the characteristic noncomputability behavior: Busy Beaver functions eventually dominate every computable function under the stated machine encoding. Here the encoding bridges matter. The theorem is about the model actually formalized, and transfer to a vendor model is mediated by explicit simulation or normalization lemmas.

7.5. Computability reading order

Read `CombinatoryLogic` first for a self-contained operational development, then the Lean core of `TuringDegrees` for quotient and oracle abstractions. Use the coverage ledger before entering the Rocq jump hierarchy. Read `BusyBeaver` last, because its interesting theorem statements are short while the important engineering lies in normal forms, exhaustive search, and certificate reachability.

8. Logic

8.1. Overview

The logic tree ranges from small propositional calculations to self-contained first-order completeness, arithmetic coding, bounded consistency, interpretability, undecidability, and a dependency-ordered port of a large external foundation library. It is also where the repository most consistently provides independent Lean and Rocq developments.

Three distinctions recur throughout the subject:

1. a metatheorem about a formal calculus versus an internal sentence of that calculus;
2. a numeralwise family, one theorem for each external natural number, versus one internally quantified uniform theorem; and
3. an independent development versus a wrapper around Mathlib or an imported formalization library.

8.2. First-order logic

[Logic/FirstOrder](#) is a reusable, mostly self-contained first-order foundation in a fixed countable language with equality and one binary function or membership-like symbol. Lean and Rocq independently define syntax, substitution, semantics, a natural-deduction calculus, and derivability.

The checked metatheory includes:

- weakening and structural transport;
- substitution and semantic replacement;
- soundness;
- a Henkin construction and Gödel completeness;
- model existence from consistency;
- equivalence of semantic satisfiability and syntactic consistency;
- compactness for the fixed language; and
- deductive transfer used by later interpretability projects.

The Henkin proof is not delegated wholesale to Mathlib. It constructs witness constants in a coded countable syntax, extends consistent theories, builds the term model, and proves the truth lemma. The project-local README is the best map from the mathematical completeness proof to the module sequence.

A separate Lean target lifts compactness to arbitrary Mathlib first-order languages. That result is useful, but it is not the independent fixed-language proof in a more polymorphic spelling. The walkthrough distinguishes the two because their dependency and trust profiles differ.

Focused builds include:

```
lake build +FirstOrder.Fo1
```


8.3. Modal logic and the Foundation port

8.3.1. FoundationPort

`Logic/FoundationPort` is a dependency-ordered Rocq port of the pinned `FormalizedFormalLogic/Foundation` source tree. Its central artifact is an exhaustive source-module ledger: every upstream module is classified as ported, intentionally excluded, replaced, or blocked, and dependencies are ordered so that a maintainer can reproduce the port rather than relying on a manually accumulated load path.

This is a conservative source port, not a claim that similarly named local Lean modules are translations of the same proofs. When an upstream theorem changes, update the source pin and ledger first; otherwise “coverage” becomes unreviewable.

8.3.2. Modal semantic core

The independent Rocq modal project formalizes Kripke frames and models, satisfaction, generated semantic correspondences, and morphism preservation. Its current scope includes:

- the general Geach correspondence and the standard T, D, B, 4, 5, Tc, and .2 instances;
- the .3 correspondence;
- the frame characterization of Löb’s axiom;
- bisimulation and bounded-morphism preservation;
- undefinability of irreflexivity;
- coarsest, finest, and transitive-closure filtrations with the semantic finite-model property; and
- the deep first-order standard translation.

The emphasis is semantic. This project should not be cited as a general Hilbert-system completeness library unless the specific target theorem and calculus are actually imported.

8.4. Propositional and equational projects

8.4.1. Shared natural deduction

`Logic/Propositional/NaturalDeduction` supplies the intuitionistic and classical calculi used by several small structural projects. It is Mathlib-free and has a standalone Lake configuration, which makes it suitable for independent checking and teaching-sized audits.

The companion projects prove:

- monotonicity/weakening of entailment;
- the principle of explosion and equivalent formulations;
- constructive commutation of adjacent universal and existential quantifiers, with finite counterexamples for analogous unique/nonexistence constructions; and
- Gödel’s theorem that no finite deterministic truth-functional matrix, in particular no three-valued matrix, characterizes intuitionistic propositional logic.

8.4.2. Sheffer stroke and single axioms

The Sheffer project gives NAND and NOR translations, proves functional completeness, and checks Nicod-style single-axiom derivations. Its role is not simply to evaluate Boolean tables: it connects the translated syntax and derivability relations and validates concrete equational proof certificates.

8.4.3. Equational certificate checker

`Logic/Equational` contains a small sound checker for equational derivations and certificate corpora for Wolfram, Meredith, Sheffer, and Huntington Boolean-algebra bases. The checker theorem is more important than any one certificate: it states that a successfully checked sequence is derivable from the chosen axioms under the formal equational rules.

Generated certificate text is therefore untrusted input to a proved checker. A reviewer should confirm both the checker's soundness theorem and the theorem connecting the final checked equation to the advertised basis result.

8.5. Peano arithmetic list coding

`Logic/PeanoArithmetic/ListCoding` develops independent Lean and Rocq codings of finite lists by natural numbers and genuine first-order PA formulas representing operations on those codes.

The current representation catalogue contains twenty-five list and number-theoretic predicates and functions, including:

- length, element access, append, flattening, and subsequences;
- sums, products, extrema, median, and unique mode;
- lexicographic order;
- one-based prime enumeration, natural powers, and prime factorization;
- canonical base digits and divisor lists; and
- the exact canonical list of distinct permutations.

A further five formulas encode ordinal notation below ε_0 : hereditary Cantor normal-form validity, ordering, addition, multiplication, and exponentiation. Thus the arithmetic formula layer contains thirty substantial represented relations rather than a handful of abstract coding lemmas.

The number-theoretic substrate includes Diophantine representations of exponentiation, tetration, and all natural hyperoperators. These are not external computability hypotheses attached at the end; they are concrete formulas with soundness and completeness theorems relative to the chosen coding.

8.5.1. Parity and audit discipline

The Lean and Rocq developments share the mathematical specification but not generated proof terms. Their encoding conventions and theorem names are mapped in project documentation. The root default build includes both the main Lean target and an audit target:

```
lake build +PAListCoding +PAListCoding.Audit
```

When extending the catalogue, add the semantic relation, the formula, both representation directions, and the cross-prover ledger entry. A formula that merely computes correctly on a few numerals is not yet a representation theorem.

8.6. Bounded consistency of PA

The bounded-consistency project formalizes finite proof search inside arithmetic. For an external natural number n , $\text{Con}_n(\text{PA})$ says that no PA proof of contradiction with code/size bounded by n exists.

8.6.1. Numeralwise scheme

Both Lean and Rocq prove the complete numeralwise scheme: for each externally supplied n , PA proves the corresponding closed bounded-consistency sentence. The construction uses an explicit proof-producing compiler, so the theorem is more informative than a metalevel observation that finite search terminates.

8.6.2. Uniform internal theorem

Lean goes further and proves a single internally quantified sentence expressing that PA proves the bounded consistency statement for every n . This is a uniform proof about the coded proof compiler, not an illicit proof of ordinary unbounded $\text{Con}(\text{PA})$. The bounded predicate still quantifies only over a finite proof region selected by its input.

The Rocq endpoint separates a remaining premise: the corresponding uniform theorem is conditional on an explicit proof-producing compiler interface, although external instances of that interface are established. This is a genuine parity difference and should not be flattened into “both provers prove the same uniform result.”

Why Gödel II is not threatened

The uniform theorem concerns a definable family of bounded searches and a verified method for producing proofs of their outcomes. It does not identify the universal closure with the standard Hilbert consistency sentence, nor does it prove that no proof of contradiction exists at every nonstandard bound in the manner required by ordinary consistency.

The root default build includes:

```
lake build +PABoundedConsistency +PABoundedConsistency.Audit
```

8.7. Interpretability

8.7.1. PA and hereditary finite set theory

[Logic/Interpretability/PAHF](#) proves a deductive bi-interpretation between PA and a finite-generation theory of hereditary finite sets. The development defines both translations, proves preservation of axioms and derivations, and establishes the round-trip equivalences required for deductive bi-interpretability. It is not merely a mutual relative-consistency argument.

PAHF is Mathlib-free and independently buildable, which makes it a useful example of how the repository packages a medium-sized self-contained logic development.

8.7.2. The ZFC-in-PA program

[Logic/Interpretability/ZFCinPA](#) attacks a much subtler target: internal PA proofs of bounded ZFC consistency/provability statements. The current development proves a large numeralwise and reduction layer but does not claim the final unconditional uniform interpretation.

The principal proved facts include:

- for every external n , ZFC proves its own bounded consistency statement $\text{Con}_n(\text{ZFC})$;
- the desired PA endpoint is reduced to eight named internal implications;

- the standard-index instances of those implications are proved; and
- the architecture explicitly handles proof translation, boundedness, and code growth.

The remaining obstacle is nonstandard: PA's universal quantifier ranges over nonstandard codes and bounds, so externally checking each standard numeral does not discharge the internal theorem. A previous five-field certificate interface was shown false; the current eight-field certificate records the additional coherence obligations exposed by that counterexample.

Accordingly, `paUniformZFCProvabilitySentence` is available as a conditional theorem whose hypotheses are the exact unresolved internal bridges. This is exemplary negative progress: the repository preserves the failed interface and its counterexample so future work cannot accidentally reintroduce the same unsound reduction.

8.8. Undecidability, finite bases, and model theory

The remaining Peano-arithmetic projects provide a broad metatheoretic layer:

- undecidability of PA, with Lean and Rocq routes adapted to their available computability libraries;
- finite-basis reductions and proof-system transport;
- Presburger arithmetic and Cooper-style quantifier elimination;
- nonisomorphic models of PA;
- theories with no finite model; and
- syntactic and deductive bridges used by bounded consistency and interpretability.

The root integration deliberately gives the undecidability and bounded-consistency projects named audit targets. Typical checks are:

```
lake build +PAUndecidable +PAUndecidable.Audit
lake build +PAFiniteBasisReduction
lake build +NoFiniteModel
```

A theorem about undecidability can depend on many hidden choices: encoding of formulas, representation of computable functions, reduction type, and consistency/soundness assumptions. The local endpoints name those choices; summaries should preserve them.

8.9. Logic reading order

A productive route is:

1. propositional natural deduction and one small structural theorem;
2. the `FirstOrder` syntax/semantics and soundness layers;
3. the Henkin construction and completeness endpoint;
4. PA list coding, beginning with one represented operation end to end;
5. bounded PA consistency, comparing numeralwise and uniform statements;
6. PAHF for a completed interpretation; and
7. ZFCinPA for a live project whose formalized obstruction is as important as its proved reductions.

9. Number theory

9.1. Overview

The number-theory tree contains two completed elementary developments, a first-order arithmetic statement project, and one very large analytic-number-theory formalization whose main conditional estimates remain open. The latter, [IntegerPoints](#), has grown far beyond the short description in the previous walkthrough and deserves to be read as an exponential-sum library rather than a single theorem stub.

9.2. Fermat’s Last Theorem for exponent four

[NumberTheory/DiophantineEquations](#) proves the exponent-four case of Fermat’s Last Theorem in Lean and Rocq under aligned public theorem names. The proof provenance differs:

- Lean imports and repackages Mathlib’s established theorem; and
- Rocq constructs the classical infinite descent from local arithmetic lemmas.

The Rocq development includes a prime-divisor toolkit, coprimality of square factors, primitive Pythagorean parametrization, parity analysis, and the odd–even descent core. Reduction theorems relate coarse and fine formulations of the descent step so that the final theorem is not tied to one awkward intermediate encoding.

This is a useful parity example. The mathematical endpoint is the same, but only the Rocq side is an independent proof of the elementary descent. “Proved in both systems” should not be expanded into “independently re-proved in both systems.”

A focused Lean build is:

```
lake build +DiophantineEquations.FermatFour
```

9.3. The primitive circle problem and exponential sums

[IntegerPoints](#) studies the number $V(x)$ of primitive lattice points in a circle. The expected main term is

$$V(x) \sim \frac{6}{\pi}x.$$

The source papers give RH-conditional error estimates, including the Zhai–Cao exponent $11/30 + \varepsilon$ and Wu’s improvement $221/608 + \varepsilon$. The current Lean development formalizes a substantial part of the machinery but does not prove those final RH-conditional theorems.

9.3.1. Source and transcription boundary

[NumberTheory/IntegerPoints/Papers](#) contains OCR-derived LaTeX transcriptions and corrections of the source papers. Wu’s original source was not fully available, and the local material marks reconstruction and correction points. These documents determine the intended formulas and numbering, but a transcribed lemma is not counted as proved until a Lean theorem imports into the proof facade.

Because analytic-number-theory OCR errors can reverse inequalities or alter exponents, the project records the source-to-formal-statement mapping. A reviewer should consult the paper transcription and the Lean declaration together.

9.3.2. Proved analytic infrastructure

The project now proves a broad collection of auxiliary results, including:

- the relevant Zhai–Cao preliminary lemmas and reductions;
- Graham–Kolesnik van der Corput lemmas through the stationary-phase and B-process layer;
- stationary phase for both curvature signs;
- finite-interval Poisson summation and Euler–Maclaurin/sawtooth identities;
- inverse-function calculus used in the transformed phase;
- the B-process theorem and closure under B-process words;
- finite Hilbert-space estimates and the required appendix inequality;
- exponent-pair admissibility restrictions and trivial seed tuples;
- Vaughan’s identity and elementary divisor decompositions; and
- several exact forms of the equation chains cited by the circle-problem papers.

The library is organized so that the analytic transformations can be reused independently of the final circle-problem application. A theorem may therefore be complete even though the headline RH estimate is not.

9.3.3. Statement-only deep estimates

Several deep literature inputs have formal declarations but no proof. The statement layer covers forms associated with Fouvry–Iwaniec, Kolesnik, Heath-Brown, Huxley, Iwaniec–Mozzochi, Hirschhorn, Littlewood–Walfisz, and Berndt–Kim–Zaharescu. These declarations are useful for type-checking the intended reduction graph and comparing exponent arithmetic, but they are not axioms injected into an unconditional endpoint.

The remaining barriers are the high-strength exponential-sum estimates and the final RH-conditional assembly. Project-local documentation classifies each cited result as proved, invoked in a proved special form, statement-only, or still needing statement correction.

The repository contains a serious formalized van der Corput/stationary-phase toolkit and many paper lemmas. It does not currently prove the Zhai–Cao or Wu main primitive-circle error terms. Any summary that quotes the final exponents must retain the words “target” or “source theorem,” unless it points to a completed `..._holds` endpoint.

9.3.4. Recommended reading route

Start with [NumberTheory/IntegerPoints/README.md](#). Then follow one complete chain: finite Poisson summation, stationary phase, B-process, and a Graham–Kolesnik lemma. Only after that inspect the paper-specific facade and exponent bookkeeping. This avoids mistaking a large import graph for completion of every cited theorem.

9.4. Exact integer sums

[NumberTheory/IntegerSums](#) proves an exact summation identity involving $\lfloor \sqrt{n} \rfloor$. The formal proof partitions the summation range into square-root blocks, evaluates each arithmetic contribution, and handles the final incomplete block. It is a compact example of turning a floor-function identity into finite interval combinatorics rather than relying on nonlinear arithmetic automation alone.

The value of this project is partly methodological: its block decomposition and cast discipline are reusable in later exact-enumeration proofs.

9.5. A rational floor orbit

[NumberTheory/RationalEnumeration](#) studies an explicitly defined floor orbit that enumerates the nonnegative rationals. The development proves both directions needed for “enumerates exactly once”: every orbit value has the stated rational form, every nonnegative rational appears, and two indices cannot represent the same rational.

Surjectivity and injectivity are separate theorems. A finite prefix or a proof of coverage alone would not establish an enumeration. The project is small enough to serve as a model for exact sequence formalizations elsewhere in the repository.

9.6. A PA sentence for an RH criterion

[NumberTheory/RiemannHypothesis/PASentence](#) defines a first-order arithmetic sentence expressing a Mertens/Littlewood-style arithmetic criterion associated with the Riemann Hypothesis. It develops the required coding of finite sums, Möbius values, and inequalities inside the PA language.

The current endpoint is a statement formalization. It does not yet prove in Lean or Rocq that the sentence is equivalent, in the standard model, to the analytic Riemann Hypothesis. That bridge would require substantial analytic and arithmetization work. The project should therefore be cited as “a PA sentence for the criterion,” not “RH formalized in PA.”

9.7. Number-theory reading order

Read Fermat four and RationalEnumeration for completed small developments. Use IntegerSums as a floor/cast case study. Approach IntegerPoints through its reusable analytic lemmas and status ledger rather than from the headline exponent. Read the RH sentence project last, with the syntax/semantics distinction from chapter 8 in mind.

10. Set theory

10.1. Overview

The set-theory tree contains a first-order ZF foundation, an equivalence between closure-style axiomatizations and ZF, and bounded-consistency developments for ZFC. It is closely connected to, but distinct from, the ZFCinPA interpretability project under [Logic](#).

10.2. A first-order ZF base

[SetTheory/ZF](#) builds Zermelo–Fraenkel set theory over the repository’s first-order syntax and semantics. It packages the membership language, standard axiom schemes, derived set constructors, and proof-system interfaces needed by closure and consistency developments.

The project is not intended as a replacement for Mathlib’s type-theoretic set universe. It treats ZF as an object theory: formulas, coded proofs, and models are explicit data, so metatheorems about derivability can be internalized later.

10.3. Closure axiomatization

[SetTheory/ClosureAxiomatization](#) proves equivalence between a closure-based axiomatization and the usual ZF package. The forward direction derives the standard axioms from closure operations; the reverse direction constructs the required closures inside ZF.

The facade is split so that the directions can be checked independently. A common focused target is:

```
lake build +ClosureAxiomatization.Forward
```

This split is useful when the forward direction is consumed by an interpretation that does not need the full reverse construction.

10.4. Bounded consistency of ZFC

[SetTheory/BoundedConsistency](#) proves the external numeralwise scheme

$$\text{ZFC} \vdash \text{Con}_n(\text{ZFC})$$

for each standard natural number n . As in the PA project, the proof is built from explicit bounded proof checking and proof generation, not from an unformalized assertion that finite search is obvious.

This theorem lives inside the ZFC object theory and should be distinguished from the PA statement that PA proves something about bounded ZFC proofs. The latter requires an interpretation and code-translation layer and is precisely where nonstandard-index problems arise.

The set-theory project proves the standard-index instances needed by the ZFCinPA reduction. It does not by itself solve the internal PA uniformity problem described in section 8.7.

10.5. Set-theory reading order

Read the fixed-language first-order foundation first, then the ZF axiom packaging. Follow one set constructor through its existence and uniqueness proofs. Next compare the two closure-axiomatization directions. Finish with bounded consistency and then cross-reference the interpretability chapter; this makes the change of object theory and metatheory explicit.

11. Ancillary material, tools, and repository boundaries

11.1. Research articles and source corpora

Several projects include full TeX articles, PDFs, OCR transcriptions, notebooks, and literature reports. These artifacts serve three legitimate purposes:

1. provenance: identifying the exact printed claim being formalized;
2. discovery: deriving a candidate identity, certificate, or proof plan; and
3. audit: recording corrections, counterexamples, and differences between the formal theorem and its source.

They do not become theorem dependencies merely by living next to Lean code. The most substantial examples are the Alaoglu–Erdős unified report, the Lazard quintic audit, the primitive-circle paper corpus, the Fabius paper formalizations, and the Ramsey paper archive.

At this snapshot the former standalone Alaoglu–Erdős literature synthesis has been absorbed into the unified report and its separate TeX/PDF pair removed. The integrated report is therefore the current review surface for both the evolving proof program and that literature crosswalk; this consolidation changes documentation topology, not the Lean theorem boundary.

11.2. Support generators

The Support, Tools, scripts, and project-local generator directories contain Python, Wolfram-language, C, shell, and PowerShell programs. Their outputs fall into three classes:

Replayed certificates. The program discovers exact data; Lean/Rocq proves that the data satisfy a sound checker. Klarner weights and many polynomial shards fit this pattern.

Native implementations. The external code is linked into evaluation of a proof-producing decision. The Ramsey counter fits this pattern and widens the trusted execution boundary.

Research evidence. The program produces numerical or symbolic observations that have not yet been connected to a theorem. A198683 exploratory partitions and some Alaoglu–Erdős scripts fit this pattern.

A walkthrough entry should state which class applies. “Generated by Python” is not enough information to assess trust.

11.3. Rocq compatibility tools

[Tools/Rocq92Compat](#) contains compatibility support for Rocq 9.2 and the repository’s mixed historical dependencies. It is infrastructure, not a mathematical theory. When a Rocq project fails after a toolchain update, inspect this layer and the `root_CoqProject` before patching every source file independently.

11.4. Vendored libraries

The `lib` tree contains focused snapshots such as the Coq Library of Undecidability Proofs and Busy Beaver certificate developments. These copies are pinned dependencies. Local wrapper theorems should make clear whether they are:

- direct re-exports;
- semantic bridges into a local model;
- assumption-preserving specializations; or
- independent proofs inspired by the vendor source.

Do not edit a vendored theorem to make a local proof pass without updating provenance and manifests. Prefer a local bridge module.

11.5. Projects moved out of the repository

The old `Shenanigans` material—kernel defects, Girard/Hurkens-style paradoxes, sanctioned escape hatches, and related experiments—has moved to a separate repository. The remaining directory is only a boundary marker or redirect. The ordinary mathematics walkthrough therefore does not treat those artifacts as current ProveIt proof targets.

Similarly, historical paths under `Oeis` may redirect to the subject-organized canonical location. New work should follow the subject tree, while compatibility paths should remain thin and explicit.

Part III.

Building, auditing, and extending the repository

12. Reproducing the surveyed workspace

12.1. Checkout and pin verification

To reproduce this walkthrough exactly, check out commit 56a402bf82a9aab119fcbe3e96b991d387b221c5 rather than the moving main branch:

```
git clone --recurse-submodules \
  https://github.com/VladimirReshetnikov/ProveIt.git
cd ProveIt
git checkout 56a402bf82a9aab119fcbe3e96b991d387b221c5
git submodule update --init --recursive
```

Then verify the two pins that control the root Lean environment:

```
cat lean-toolchain
grep -n 'mathlib' lake-manifest.json
```

At the surveyed commit, the intended pair is Lean 4.32.0 and Mathlib v4.32.0. The manifest, not an ambient global installation, determines the dependency revision.

On Windows PowerShell the same checkout can be made with ordinary Git for Windows; no path rewriting is required by the repository itself. Long generated module names make enabling long paths prudent:

```
git config --global core.longpaths true
```

12.2. Mathlib cache and root build

Fetch the binary cache before compiling:

```
lake exe cache get
lake build
```

The cache command is not part of theorem checking. It downloads precompiled Mathlib artifacts matching the manifest. A cache miss increases build time but should not change theorem status.

The repository's local guidance generally disables Lean's worker pool for expensive targets:

```
LEAN_NUM_THREADS=0 lake build +PolynomialFormulas
```

or in PowerShell:

```
$env:LEAN_NUM_THREADS = '0'
lake build +PolynomialFormulas
```

This setting does not disable native code used by `native_decide`; it controls Lean's elaboration worker pool.

12.3. Choosing the right Lean command

There are four useful checking granularities.

12.3.1. Default integration build

```
lake build
```

This builds the seven default targets named in chapter 2. It is the right final integration check but often the wrong edit-compile loop.

12.3.2. Named library or facade

```
lake build FabiusFunction
lake build +SquaredSquare
lake build +TuringDegrees +TuringDegrees.Audit
```

Use the exact library names from `lakefile.toml`. The leading plus requests an explicit target in Lake’s command syntax.

12.3.3. Direct module elaboration

When a dotted name is not a declared target, invoke Lean directly through Lake’s environment:

```
lake env lean path/to/Module.lean
```

For modules that create object artifacts or whose directory is read-only, provide explicit output paths as needed. Direct elaboration is also the best way to verify that a file omitted by all facades still type-checks.

12.3.4. Temporary import probe

For a complicated import surface, create a scratch file outside the repository source tree:

```
import Target.Facade
#print axioms Target.main_theorem
#check Target.secondary_theorem
```

Then run it with `lake env lean`. This tests public reachability and theorem names without modifying a committed facade.

12.4. Reading Lake’s build plan

A zero exit code is meaningful only if Lake scheduled the desired module. Use verbose output or inspect the build directory when uncertain:

```
lake -v build +TargetName
```

Warning signs of a silent no-op include no compilation lines, no updated object/olean files, and a target name that does not occur in `lakefile.toml`. This matters especially in libraries declared with a single root facade rather than a glob.

12.5. Rocq aggregate build

The root Rocq workspace requires its submodules and, for some projects, prebuilding a focused upstream dependency. The documented aggregate route is:

```
git submodule update --init \
  lib/Coq-Synthetic-Computability \
  lib/MathComp-Abel
opam install --yes --deps-only \
  ./lib/MathComp-Abel/coq-mathcomp-abel.opam
pwsh -NoProfile -File \
  Computability/TuringDegrees/Coq/BuildSyntheticComputability.ps1
rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq
```

For review work, sequential `rocq compile` or `coqc` commands are often easier to diagnose than the aggregate makefile. Preserve the `-Q` mappings from `_CoqProject`; compiling a file under a different logical path can create incompatible library names even when source text is unchanged.

12.6. External tools are optional until a target says otherwise

Python, a Wolfram engine, a C compiler, PowerShell, and document tools occur in support workflows. They are not globally required for every theorem. The target-specific distinction is:

- ordinary Lean/Rocq proofs need only their declared proof-assistant dependencies;
- regeneration workflows need the project's support language;
- native certificates with FFI need a C toolchain and the dedicated link target;
- article builds need the TeX packages recorded by that article; and
- source-audit workflows may need PDF rendering or exact-computation tools but do not alter the kernel theorem.

13. Auditing theorem status and trust

13.1. Start from an endpoint theorem

A reliable audit runs backward from a theorem a downstream user will cite. Record:

1. fully qualified theorem name;
2. statement after implicit arguments are displayed;
3. source file and facade that exports it;
4. assumptions reported by the proof assistant;
5. imported conditional propositions;
6. computation mechanism used in the proof; and
7. any source paper or generated certificate represented by constants in the statement.

Starting from a README headline and searching for a similarly named declaration is riskier: many projects intentionally have a proposition-valued source claim and a separate `..._holds` theorem.

13.2. Lean assumption audits

Use `##print axioms` on public endpoints:

```
import Project.Facade
#print axioms Project.main_theorem
```

The output names logical constants in the theorem's proof term. Standard appearances of `propext`, `Classical.choice`, or `Quot.sound` are not admissions, but they may matter to a constructive or quotient-free consumer.

Also search the relevant source closure for:

```
rg -n '\bsorry\b|admit|axiom|opaque|unsafe|native_decide|implemented_by|extern' \
path/to/project
```

This is a triage command, not a proof of cleanliness. For example, an `unsafe` support definition may be unreachable from a theorem; conversely, an imported axiom can be hidden outside the searched directory. The endpoint's axiom report and import graph remain primary.

13.3. Rocq assumption audits

Use both source-level and kernel-level checks:

```
Print Assumptions MainTheorem.
```

and

```
coqchk -silent -o -Q path LogicalRoot LogicalRoot.Module
```

`Print Assumptions` explains the theorem's constants; `coqchk` rechecks compiled libraries. Projects that use classical real analysis may report standard library axioms, while syntactic projects often close with none. The project README should state the expected footprint.

13.4. Audit a generated certificate in layers

For a proved checker with generated data:

1. inspect the abstract predicate the data must satisfy;
2. inspect the checker and its soundness theorem;
3. identify the concrete committed data object;
4. inspect the theorem that runs or reduces the checker on that object;
5. inspect the theorem translating checker success into the advertised mathematics; and
6. regenerate independently only after the formal path is understood.

This order prevents a common mistake: spending days reviewing a generator while missing that the final theorem checks a different table or that the generated shard is not imported.

13.5. Audit native decision separately

For `native_decide`, ask two independent questions:

Logical question. Is the decidable proposition exactly the desired theorem, and are all equivalence bridges proved?

Execution question. Which definitions execute after compiler extraction? Do they contain `implemented_by`, `extern`, unsafe opacity, hash-table assumptions, or platform-dependent integer operations?

The A002845 sparse counts and the Ramsey finite counts both use native decision, but only the latter crosses an FFI boundary. A single trust label for both would erase the relevant difference.

13.6. Audit paper catalogues

For a paper-scale project:

1. locate the source transcription and numbered-claim crosswalk;
2. compare the exact formal statement with the source, including domains and endpoint conventions;
3. verify whether the statement is a `def ... : Prop`, theorem, or axiom;
4. locate the companion `..._holds` theorem;
5. confirm that the public proof facade imports it; and
6. read any errata note before treating a mismatch as a formalization bug.

The Ramsey and Fabius projects use this pattern extensively. It lets the repository retain a complete statement catalogue while proof coverage grows incrementally.

13.7. Audit conditional endpoints

Rewrite a conditional theorem in ordinary mathematics before citing it. For example:

Assuming every coefficient in the conjectured support is nonzero, the A290268 count equals the closed form.

is informative. “Lean proves the A290268 formula” is not. Likewise, the weight-two Alaoglu–Erdős collapse theorem completely proves the algebra after a linear-independence premise; it does not prove that open transcendence premise.

14. Debugging large formal developments

14.1. Classify the failure before editing

Most failures fall into one of five classes:

Reachability. The intended file is not in the build plan or facade.

Elaboration. Names, implicit arguments, coercions, or typeclass instances no longer resolve.

Resource. Heartbeats, recursion depth, native compilation, linker memory, or generated source size exceeds a limit.

Certificate mismatch. Generated data no longer matches the checker or semantic bridge.

Mathematical gap. The theorem genuinely needs a new lemma or the statement is false.

Treating every failure as the last class creates unnecessary proof rewrites; treating every failure as tooling hides real counterexamples.

14.2. Minimize the import surface

Create a scratch module that imports the narrowest file exhibiting the problem. Large facades can introduce notation, simp lemmas, instances, and heartbeat costs unrelated to the target theorem. Conversely, a narrow module may fail because it accidentally depended on an import supplied only by a broad facade; that is valuable dependency information.

For generated shards, compile one shard and its abstract checker before compiling the endpoint aggregate. For paper suites, compile the statement module before the proof module.

14.3. Coercion and normalization failures

Many projects cross `Nat`, `Int`, `Rat`, `Real`, polynomials, and algebraic extensions. A robust debugging sequence is:

1. expose the target with `show` or `change`;
2. normalize casts with a narrowly chosen lemma or `norm_cast`;
3. separate nonzero/positivity side conditions;
4. use `ring` or `ring_nf` only after transcendental or order-sensitive rewrites; and
5. keep the semantic equality separate from the finite certificate equality.

This pattern is visible in the trigonometric, finite power, stationary-phase, and polynomial projects.

14.4. Resource controls are part of the module contract

Large modules intentionally set options such as `maxRecDepth`, `maxHeartbeats`, or linter exceptions. Do not remove them as cosmetic clutter. Before increasing a limit:

- determine whether the blow-up is elaboration, reduction, or native compilation;
- verify that a facade has not imported a huge certificate unnecessarily;
- split tables or proof chains at semantic boundaries; and
- compare with the project-local standalone workspace, which may have tuned options.

A tenfold heartbeat increase can conceal an accidental quadratic rewrite; a shard split can make the proof both faster and more reviewable.

14.5. Generated-data drift

When regeneration changes a certificate:

1. preserve the old generated file long enough to diff semantic records, not formatting;
2. run the exact external replay or regression script;
3. compile the checker on the new data;
4. verify endpoint cardinalities or hashes only as secondary diagnostics; and
5. update provenance, generator version, and any status ledger in the same change.

Never hand-edit a generated coefficient solely to make decide pass. Either the generator, serialization, checker semantics, or underlying mathematics has changed; the repository should record which.

14.6. Native and linker failures

For the Ramsey target, distinguish:

- compilation of the C source;
- construction of the shared object or library;
- loading through the shim;
- resolution of `lean_ramsey_fast_reflected_m`; and
- execution of `native_decide`.

A proof elaboration failure before native evaluation is not an FFI problem. A missing symbol after successful Lean compilation is not a theorem problem. Keep the small shim target precisely because it localizes this boundary.

14.7. False statements and failed interfaces

When a proposed lemma is false, preserve the counterexample in the formal development when it teaches something structural. The Lazard audit, Fabius errata modules, and ZFCinPA certificate redesign all benefit from this practice. A named refutation prevents a later contributor or automated agent from repeatedly rebuilding the same invalid bridge.

15. Adding or expanding a project

15.1. Choose the canonical subject path

Place new work under the mathematical subject hierarchy, not under a generic OEIS or experiments root. A typical layout is:

```
Subject/Project/  
  README.md  
  Lean/  
    Project.lean  
    Project/...  
  Coq/          # when an independent Rocq development exists  
  Papers/       # exact sources/transcriptions  
  Support/      # generators and external replay  
  Research/     # explicitly non-theorem work
```

Not every project needs every directory. Empty symmetry is less useful than a clear proof boundary.

15.2. Define one public facade

The facade should import the supported theorem surface, not every research file. A project with statement catalogues and proofs may provide two facades. A project with huge optional certificates should keep them behind an explicit target.

Add the library and source roots to [lakefile.toml](#). Then decide independently whether it belongs in:

- the broad [ProveIt.lean](#) facade;
- the seven-target default build;
- a dedicated audit target; and
- the root README's focused-build examples.

These are policy choices, not automatic consequences of creating a file.

15.3. Write a status ledger before the project becomes large

A useful README answers:

1. What is the canonical mathematical statement?
2. Which endpoint theorems are complete?
3. Which results are conditional, statement-only, finite, or heuristic?
4. Which source version is being formalized?
5. Which commands actually reach each endpoint?
6. What is the expected assumption footprint?
7. Which support tools are reproducibility aids versus execution dependencies?

The A198683, IntegerPoints, TuringDegrees, Lazard, and ZFCinPA ledgers demonstrate why this becomes indispensable after dozens of modules.

15.4. Design certificate interfaces for failure

A certificate record should be strong enough that a successful check implies the intended theorem and weak enough that valid certificates can be produced. Before generating a million coefficients, test the interface on:

- small positive instances;
- adversarial malformed inputs;
- boundary cases such as repeated roots, zero denominators, branch cuts, and nonstandard indices; and
- any known counterexample to a previous interface.

Prove checker soundness before optimizing the generator. If performance later requires an executable replacement, expose that trust expansion explicitly.

15.5. Add audits as build products

For Lean, create an audit module that imports the public endpoint and prints its axioms. For Rocq, include `Print Assumptions` and a `coqchk` command. For a paper catalogue, add a machine-readable or at least tabular source-claim crosswalk. For generated data, add deterministic regeneration and exact external replay.

An audit hidden in a developer's shell history is not a maintainable artifact.

15.6. Keep documentation reachability-tested

Every command in a README should be periodically executed from a clean checkout. Every theorem name in a status table should be imported by the documented facade. Every link should point to the canonical path. The old walkthrough became obsolete primarily because project content evolved faster than its manually summarized surface; the antidote is to tie prose to facades, ledgers, and pins.

16. Current formalization frontiers

16.1. Completed headline developments

At the surveyed commit, the following are representative completed endpoints rather than mere statement catalogues:

- the explicit Jacobian counterexample and stabilization families;
- exact radical formulas through degree four;
- three Abel–Ruffini formalization routes;
- a primitive-recursive decision procedure for integer quintic solvability by radicals;
- a computable sextic decision procedure with a Rocq partial-recursive realization;
- the three-base integral-exponent theorem for bases 2, 3, and 5;
- the exact tiny exponent-tower floor;
- the core Fabius construction, dyadic evaluator, paper formalizations, and corrected periodic asymptotic;
- A158415 at size 15, A198683 through size 7, and substantial exact natural/ordinal power-tower tables;
- the Klarner upper bound 4.5235;
- Duijvestijn’s 21-square dissection and the lower bound seven;
- first-order soundness, completeness, and compactness in the fixed language;
- PA list coding and the represented arithmetic/list catalogue;
- the numeralwise and Lean-uniform bounded PA consistency results;
- PA–hereditarily-finite-set bi-interpretability;
- the principal SK/SKI/Iota operational universality chain;
- the Lean core of Turing degrees and many Rocq jump/hierarchy results under named premises; and
- Fermat four, exact integer sums, and rational enumeration.

This list is illustrative, not a replacement for theorem names. “Completed” means the advertised endpoint is exported with its stated hypotheses; it does not imply identical proof provenance in Lean and Rocq.

Area	Formalized frontier
------	---------------------

16.2. Precisely isolated open barriers

Table 16.1.: Major current barriers and their formalized reductions.

Area	Formalized frontier
Lazard quintic	Large proof-carrying symbolic pipeline and corrected invariant identities; automatic completion of the full classical solvable-quintic endpoint remains active work.
Alaoglu–Erdős	Counterexamples excluded below $2^x < 16384$; zero-density and extensive structural reductions proved; Four Exponentials and weight-two logarithmic independence remain open transcendence inputs.
A290268	Closed form, recurrence, generating function, support size, and structural inclusion proved; nonvanishing throughout the conjectured support remains.
A198683 at $n = 12$	Exact candidate partitions and several separation lemmas proved; the final partition/overflow no-miracles obligation still controls the unconditional value.
Squared-square minimum	Existence at 21 and lower bound seven proved; verified completeness of Duijvestijn’s exhaustive order-20 search remains.
Gowers–Szemerédi	Complete claim catalogue and substantial <code>_holds</code> layer; remaining numbered claims are visible by absence of proof endpoints.
Primitive circle problem	Poisson, stationary phase, B-process, exponent-pair and paper reductions proved in substantial part; deepest exponential-sum estimates and final RH-conditional bounds remain.
ZFC in PA	Standard numeral instances and eight-implication reduction proved; arbitrary internal/nonstandard successor implications remain.
Lean BB4	Transition-normal-form reduction sound; exhaustive equality of the root classification remains conditional.
Turing degrees	Broad core and several Rocq hierarchy/existence results; many classical deep existence and classification theorems deliberately absent or parameterized.

16.3. Architectural strengths

The repository’s strongest recurring design choices are:

- theorem statements and computational representations are connected by explicit bridges;
- false source claims and failed certificate interfaces are retained as formal refutations;
- conditional conclusions keep their hypotheses in signatures;
- large generated calculations are split into checker, data, and endpoint layers;
- independent Lean/Rocq developments share specifications rather than proof artifacts; and
- heavy targets are opt-in rather than making every integration build unusable.

16.4. Architectural pressure points

The same growth creates maintenance pressure:

- the root README and subject READMEs can lag newly merged trees, as happened with Ramsey;
- the broad facade and default targets are different reachability surfaces and are easy to confuse;

- generated polynomial and analytic-research forests need machine-readable status extraction;
- project-local Lake files and the root workspace can drift in options or dependencies;
- native implementations need explicit extensional-validation tests even when the theorem boundary intentionally trusts them; and
- paper catalogues need automated detection of definitions lacking imported `_holds` theorems.

A future documentation generator could read `lakefile.toml`, facade imports, audit modules, and structured status tables to produce much of the inventory in this walkthrough. Human prose would then focus on mathematical architecture and open barriers rather than manually tracking hundreds of paths.

Part IV.

Reference

A. Command crib sheet

A.1. Root and broad subject builds

```
lake exe cache get
lake build
lake build ProveIt
lake build JacobianConjecture
lake build +PolynomialFormulas
lake build FabiusFunction
lake build +IntegerPoints
lake build +PowerTowers
lake build +A158415
lake build +A290268
lake build +KlarnarConstant
lake build +SquaredSquare
lake build +GowersSzemerédi
lake build +RamseyPaperCertificates
lake build +CombinatoryLogic
lake build +TuringDegrees +TuringDegrees.Audit
```

Not every command above has the same cost or external-tool requirement. The Ramsey certificate target needs its native/C build path; large PolynomialFormulas and Busy Beaver targets should be run serially.

A.2. Logic and number-theory targets

```
lake build +FirstOrder.Fol
lake build +ArbitraryLanguageCompactness
lake build +ShefferStroke.Sheffer
lake build +PAListCoding +PAListCoding.Audit
lake build +PAFiniteBasisReduction
lake build +PAUndecidable +PAUndecidable.Audit
lake build +PABoundedConsistency +PABoundedConsistency.Audit
lake build +NoFiniteModel
lake build +DiophantineEquations.FermatFour
lake build +IntegerSums
lake build +RationalEnumeration
lake build +ClosureAxiomatization.Forward
```

Exact target spelling is determined by [lakefile.toml](#); when a command is rejected or silently schedules nothing, inspect the current manifest rather than guessing a dotted target.

A.3. Audits and searches

```
rg -n '\bsorry\b|admit|axiom' Algebra Analysis Combinatorics \
  Computability Logic NumberTheory SetTheory
rg -n 'native_decide|implemented_by|extern|unsafe' .
rg -n '^import ' path/to/Facade.lean
lake -v build +Target
lake env lean /tmp/Audit.lean
```

For Rocq:

```
rocq makefile -f _CoqProject -o Makefile.coq
make -f Makefile.coq
coqchk -silent -o -Q path LogicalRoot LogicalRoot.Audit
```

B. Project and facade index

Table B.1.: Canonical project locations and first files to read.

Project	Canonical directory	First review surface
Jacobian counterexample	Algebra/JacobianConjecture	README, Lean facade, explicit map and Jacobian determinant theorem.
Polynomial formulas	Algebra/PolynomialFormulas	README, facade, then the relevant degree/decision/Lazard subfacade.
Trigonometric identities	Analysis/TrigonometricIdentities	Two small theorem modules.
Exponential identities	Analysis/ExponentialIdentities	README and facade; select either three-base, tiny tower, or one two-base branch.
Fabius function	Analysis/FabiusFunction	README, facade, construction, dyadic evaluator, q-binomial/global-series and asymptotic endpoints.
A290268	Combinatorics/DerivativeExpansions/A290268	Facade and Main.lean.
Power towers	Combinatorics/ExpressionEnumeration/PowerTowers	Facade, shared core, sequence module, A198683 research ledger when relevant.
A158415	Combinatorics/ExpressionEnumeration/RadicalExpressions/A158415	Core, facade, size-15 result and order certificate.
Klarner constant	Combinatorics/Polyominoes/KlarnerConstant	README and geometric-completeness endpoint.
Squared square	Combinatorics/SquaredSquare	README, facade, Duijvestijn, minimality, minimal-order reduction.
Ramsey papers	Combinatorics/Ramsey	Common layer, one statement facade, companion proofs, certificate boundary.
Busy Beaver	Computability/BusyBeaver	Subject README, model/measure definitions, selected opt-in classifier.
Combinatory logic	Computability/CombinatoryLogic	README, reduction core, one compiler, audit.
Turing degrees	Computability/TuringDegrees	Coverage ledger, Lean facade, Rocq theorem with explicit premises.
First-order logic	Logic/FirstOrder	README, syntax/semantics, soundness, Henkin/completeness.
Modal logic	Logic/Modal	Rocq facade and semantic correspondence modules.
PA list coding	Logic/PeanoArithmetic/ListCoding	README, coding core, one representation proof, audit.
Bounded PA consistency	Logic/PeanoArithmetic/BoundedConsistency	README, numeralwise compiler, uniform Lean endpoint, Rocq premise boundary.
PAHF	Logic/Interpretability/PAHF	README, translations, deductive preservation, round-trip theorem.
ZFC in PA	Logic/Interpretability/ZFCinPA	README, main reduction facade, eight implication interfaces, refutation of old certificate.
Primitive circle problem	NumberTheory/IntegerPoints	README, paper crosswalk, proved analytic facade, remaining-statement ledger.
ZF and closure	SetTheory/ZF, SetTheory/ClosureAxiomatization	Object-theory facade, then forward/reverse closure theorems.
Bounded ZFC consistency	SetTheory/BoundedConsistency	Numeralwise endpoint and audit.

C. Trust-boundary checklist

Before describing a result as machine-checked, answer all applicable questions:

1. Is the advertised declaration a theorem, or only a proposition-valued definition?
2. Does the documented facade import the proof theorem?
3. What hypotheses remain in the theorem statement?
4. What does `##print axioms` or `Print Assumptions` report?
5. Does the proof use `decide`, `native_decide`, `vm_compute`, or a proved checker?
6. Does native execution encounter `implemented_by`, `extern`, or `unsafe opacity`?
7. Is generated data rechecked semantically, or is only a hash/cardinality checked?
8. Is a source statement manually transcribed, and where are corrections recorded?
9. Is a vendor theorem imported directly or transferred through a proved semantic bridge?
10. Does the cited build command actually schedule the module?
11. Are Lean and Rocq endpoints independent, wrappers, or only partially aligned?
12. Is the result universal, finite, numeralwise, conditional, or heuristic?

A concise trustworthy status line can then be written in the form:

Lean proves theorem T at commit 56a402bf; the endpoint assumes H , uses `native_decide` over a proved finite specification, executes no foreign replacement, and its axiom audit reports only standard classical principles.

D. What changed since the previous walkthrough

The 685-commit interval is too large for a file-by-file changelog. The changes that materially alter a reader's mental model are:

- `ExponentialIdentities` expanded into a large Alaoglu–Erdős research library, including exact finite exclusion through $2^x < 16384$, density-zero results, Four Exponentials reduction, radical and valuation reformulations, determinant/tropical branches, the finite prime-necklace smooth-level classification, and the weight-two collapse endpoint.
- `FabiusFunction` became a completed large formalization with exact dyadic computation, complete paper ports, finite and globally convergent q -binomial formulas, and a corrected periodic asymptotic theory.
- `IntegerPoints` acquired a substantial reusable exponential-sum and stationary-phase development, while retaining an explicit statement-only frontier for the deepest estimates.
- `Combinatorics/Ramsey` was added and merged into the root facade, with several paper catalogues/proof layers, exact finite counts through 20, a dedicated C-backed native certificate target, and newly completed Section 9 random-restriction endpoints.
- Polynomial radical-solvability work expanded from formulas and impossibility results to executable quintic and sextic decision architectures and a much larger proof-carrying Lazard development.
- `A158415` now has an exact size-15 theorem, $a(15) = 791$.
- `A290268` now isolates the universal conjecture to a single support-nonvanishing direction after proving the closed-form and structural layers.
- The `A198683` size-12 research program acquired sharper formal partitions and separation lemmas while preserving the final open overflow condition.
- `SquaredSquare` now packages the sharp order-21 claim as an exact equivalence with the unformalized exhaustive-search proposition, alongside the completed bracket $7 \leq m \leq 21$.
- `ZFCinPA` replaced a refuted five-field certificate interface by an eight-field architecture that exposes the nonstandard-index obligations.
- Documentation and build topology became more important: the broad root facade, default targets, audit targets, native link targets, and project-local workspaces now form distinct verification surfaces.

E. Glossary of repository terms

Audit module. A module whose purpose is to import endpoints and report assumptions or other integrity facts, rather than add mathematics.

Certificate. Finite data intended to witness a proposition and consumed by a verified or explicitly trusted checker.

Facade. A small public module importing the supported surface of a project.

Kernel certificate. A finite proposition reduced by the proof assistant’s kernel, for example with Lean `decide`.

Native certificate. A proposition discharged through compiled evaluation such as Lean `native_decide`; its execution boundary may be wider than the kernel.

Numeralwise scheme. One externally generated theorem for each standard natural number, not a single internally quantified theorem.

Paper catalogue. Formal proposition-valued definitions corresponding to numbered claims in a source paper.

Proof facade. A facade that imports theorems establishing catalogue claims, often named with `_holds`.

Research ledger. Documentation classifying claims as proved, conditional, data-supported, refuted, or heuristic.

Semantic bridge. A theorem connecting a computation-facing representation to the canonical mathematical definition.

Statement-only. A formal well-typed proposition with no proof theorem currently exported.

Support generator. External code that discovers or serializes proof data but is not necessarily part of the theorem’s trusted execution.

F. Closing perspective

The repository is best understood not as a bag of Lean files but as a federation of proof architectures. A twenty-line trigonometric theorem, a million-character research article, a C-backed reflected counter, a Rocq compiler correctness proof, and a generated quintic invariant can all be legitimate formal artifacts, but they answer different questions and rely on different evidence.

The maintainable unit is therefore the complete chain

$$\begin{aligned} \text{source claim} &\longrightarrow \text{canonical statement} \longrightarrow \text{representation or certificate} \\ &\longrightarrow \text{checker/proof} \longrightarrow \text{public endpoint} \longrightarrow \text{audit and build target.} \end{aligned}$$

The current repository is strongest where every arrow is named. Its most interesting open projects are likewise strongest where the missing arrow has been isolated as a precise proposition rather than hidden behind an optimistic status label.

This walkthrough is pinned to 56a402bf82a9aab119fcbe3e96b991d387b221c5. Future updates should begin by comparing that commit with the new head, then regenerate the library/facade inventory before editing mathematical prose. That procedure is less glamorous than discovering a theorem, but it is what keeps a fast-moving formalization repository intelligible.